Financial Risk Assessment (NLP Coursework Notebook)

```
# Google drive set up
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
# Installing some of the required libraries
!pip install datasets
!pip install nlpaug
!pip install keybert sentence-transformers
!pip install yake
```

Show hidden output

```
# Loading the dataset (from HuggingFace)
from datasets import load_dataset
ds = load_dataset("gretelai/gretel-financial-risk-analysis-v1")
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
README.md: 100%  7.44k/7.44k [00:00<00:00, 239kB/s]
train-00000-of-00001.parquet: 100%  1.78M/1.78M [00:00<00:00, 16.1MB/s]
test-00000-of-00001.parquet: 100%  458k/458k [00:00<00:00, 1.22MB/s]
Generating train split: 100%  827/827 [00:00<00:00, 6132.72 examples/s]
Generating test split: 100%  207/207 [00:00<00:00, 4444.44 examples/s]
```

```
# Checking the first row
ds['train'][0]
```

```
{'input': '"Item 8.01. Other Events.\n\nOn March 21, 2023, the Company entered into an agreement with its wholly-owned subsidiary, F5, Inc. (F5) to merge F5 with a subsidiary of the Company. The merger is expected to be completed in the second quarter of 2023, subject to the satisfaction of customary closing conditions, including the receipt of regulatory approvals from relevant authorities.\n\nThe merger consideration will be paid in cash, with the Company paying F5 stockholders approximately $4.4 billion, or $23.00 per share, in the aggregate. This price represents a premium of approximately 25% to the closing price of F5\'s common stock on the trading day preceding the announcement of the merger. The merger consideration will be funded with the Company\'s existing cash reserves, which totaled approximately $8.2 billion as of December 31, 2022. The Company has sufficient liquidity to fund the merger consideration and does not anticipate the need to raise additional debt or equity financing in connection with the transaction.\n\nIn addition to the merger consideration, the Company will also assume F5\'s outstanding debt of approximately $1.4 billion, which consists of senior notes with a weighted average interest rate of 4.2% and a weighted average maturity of 7.3 years. The assumption of F5\'s debt is expected to increase the Company\'s total debt to approximately $6.5 billion, pro forma for the merger. However, the Company believes that the merger will generate significant cash flow synergies and improve its overall financial profile, enabling it to manage its increased debt burden effectively.\n\nThe merger is subject to customary closing conditions, including the receipt of regulatory approvals from the Federal Trade Commission (FTC) and the Department of Justice (DOJ) under the Hart-Scott-Rodino Antitrust Improvements Act (HSR Act). The Company has filed the required notifications with the FTC and DOJ and is cooperating with their review of the transaction. While there can be
```

no assurance that the regulatory approvals will be obtained on a timely basis, or at all, the Company believes that the merger will not raise significant competitive concerns and expects to receive the necessary approvals within the anticipated timeframe.\n\nIn connection with the merger, the Company will file a Current Report on Form 8-K, which will include the merger agreement and related exhibits, with the Securities and Exchange Commission (the SEC). The merger agreement will provide additional details about the terms and conditions of the transaction, including the merger consideration, the assumption of F5\'s debt, and the post-merger organizational structure. The Company will also file with the SEC a Current Report on Form 8-K announcing the closing of the merger, which will include updated financial information and other relevant details about the transaction.\n\nThe Company expects the merger to generate significant strategic and financial benefits, including enhanced scale and competitiveness, improved operational efficiency, and increased cash flow. The merger is also expected to create opportunities for growth and expansion in new markets and geographies, as well as increased investment in research and development, sales and marketing, and customer support. The Company is committed to ensuring a smooth transition and integration of F5\'s business and operations, with minimal disruption to customers, employees, and other stakeholders."',
'output': {'analysis': 'Assuming $1.4B debt with 4.2% interest rate; merger subject to regulatory approvals',
'critical_dates': ['2023-06-30'],
'financial_impact': {'amount': 1400.0,
'recurring': False,
'timeframe': None},
'key_metrics': {'debt_outstanding': 6500.0,
'hedge_ratio': None,
'interest_rate': 4.2,
'tax_exposure': None},
'risk_categories': ['DEBT', 'LIQUIDITY', 'REGULATORY'],
'risk_severity': 'HIGH'},
'risk_severity': 'HIGH',
'risk_categories': ['DEBT', 'LIQUIDITY', 'REGULATORY'],
'text_length': 3457,
'__index_level_0__': 94}

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import re
from wordcloud import WordCloud
from collections import Counter
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer


# Converting to pandas dataFrame for easier manipulation
df = pd.DataFrame(ds['train'])
test_df = pd.DataFrame(ds['test']) # Test set is untouched until final evaluation


# Downloading some necessary nltk operations for initial analyses
nltk.download('stopwords')
nltk.download('wordnet')
os.makedirs('/root/nltk_data', exist_ok=True)
# Download punkt_tab resource to that directory
nltk.download('punkt_tab', download_dir='/root/nltk_data')
# Add the downloaded directory to nltk's search path
nltk.data.path.append('/root/nltk_data')
# Import and use word_tokenize as before
from nltk.tokenize import word_tokenize
nltk.download('punkt')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
True
```

Exploratory Analysis

```
df.head()
```

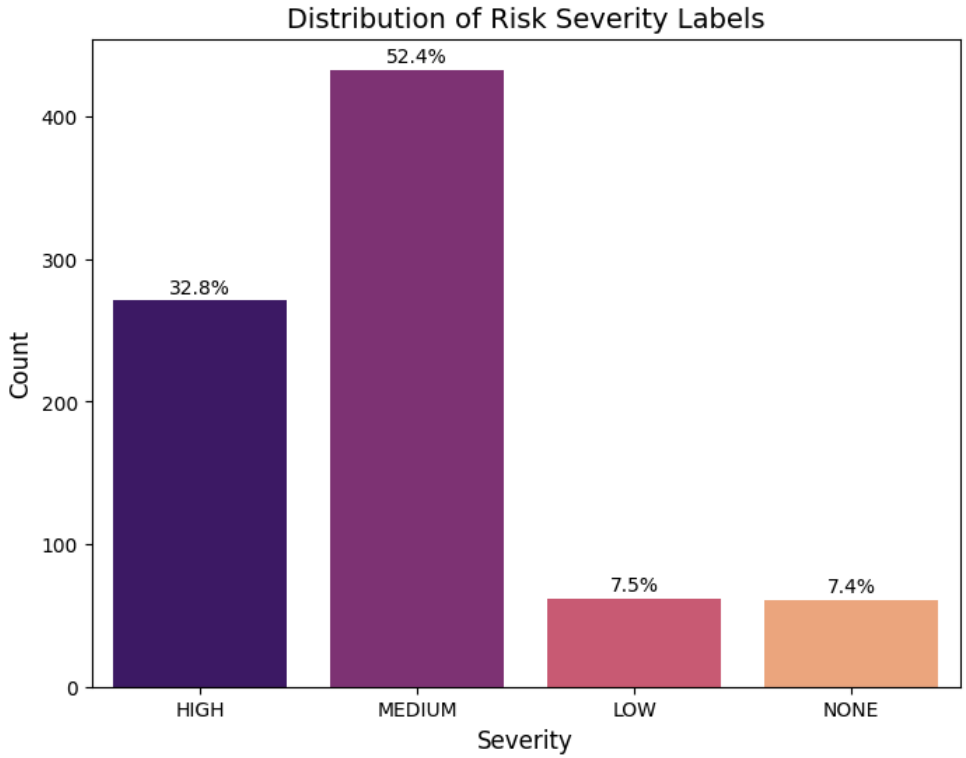
	input	output	risk_severity	risk_categories	text_length	__index_level_0__
0	"Item 8.01. Other Events.\n\nOn March 21, 2023...	{'analysis': 'Assuming $1.4B debt with 4.2% in...	HIGH	[DEBT, LIQUIDITY, REGULATORY]	3457	94
1	", and to the extent that the financial market...	{'analysis': 'Significant disruptions to globa...	HIGH	[OPERATIONAL, MARKET, LABOR]	5494	728
2	"Item 8.01\nDate: September 23, 2022\n\nExhibi...	{'analysis': 'Tentative labor agreement with U...	MEDIUM	[LABOR]	3771	1
3	"the Company's financial condition, results of...	{'analysis': 'High debt exposure ($1.2B) with ...	HIGH	[DEBT, INTEREST_RATE]	7039	864
4	Item 8.01 Other Events\n\nOn April 15, 2022, t...	{'analysis': 'Settlement payment of $400,000 t...	LOW	[LEGAL]	3374	837

Next steps: Generate code with df | View recommended plots | New interactive sheet

```
# Risk Severity Distribution
severity_counts = df['risk_severity'].value_counts()
severity_percentages = df['risk_severity'].value_counts(normalize=True) * 100
print("Severity Distribution (Counts):")
print(severity_counts)
print("\nSeverity Distribution (Percentages):")
print(severity_percentages)
```

```
Severity Distribution (Counts):
risk_severity
MEDIUM    433
HIGH      271
LOW        62
NONE       61
Name: count, dtype: int64

Severity Distribution (Percentages):
risk_severity
MEDIUM    52.357920
HIGH      32.769045
LOW        7.496977
NONE       7.376058
Name: proportion, dtype: float64
```

```
# Severity Distribution Bar Plot
severity_order = ['HIGH', 'MEDIUM', 'LOW', 'NONE']

plt.figure(figsize=(8, 6))
sns.countplot(data=df, x='risk_severity', hue='risk_severity', order=severity_order, palette='magma')
plt.title('Distribution of Risk Severity Labels', fontsize=14)
plt.xlabel('Severity', fontsize=12)
plt.ylabel('Count', fontsize=12)
for i, severity in enumerate(severity_order):
    count = df['risk_severity'].value_counts().reindex(severity_order)[severity]
    percent = (count / len(df)) * 100
    plt.text(i, count + 5, f'{percent:.1f}%', ha='center', fontsize=10)
plt.show()
```

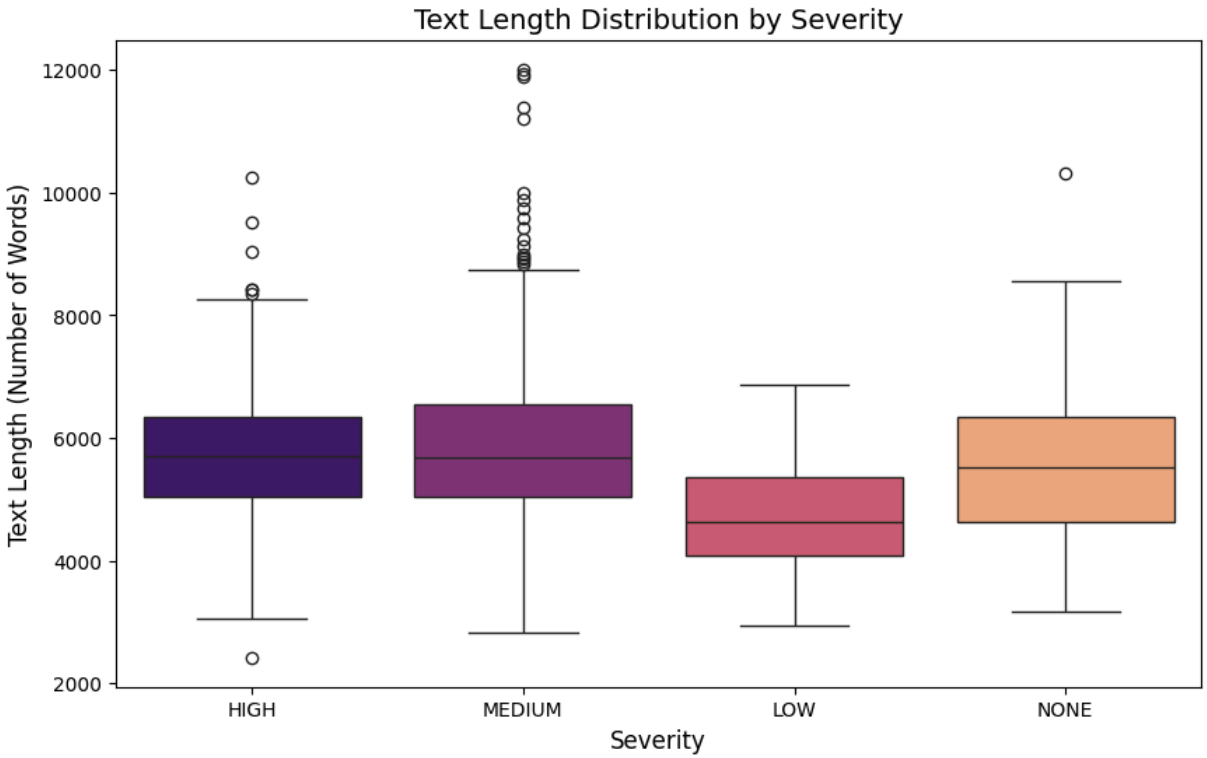


Medium (52.4%) and High (32.8%) severity classes dominate, with Low and None at only ~7.4% each. Oversampling/Text augmentation may help improve performance later on.

```
# Text Length Analysis (Statistics)
print("\nText Length Statistics:")
print(df['text_length'].describe())
```

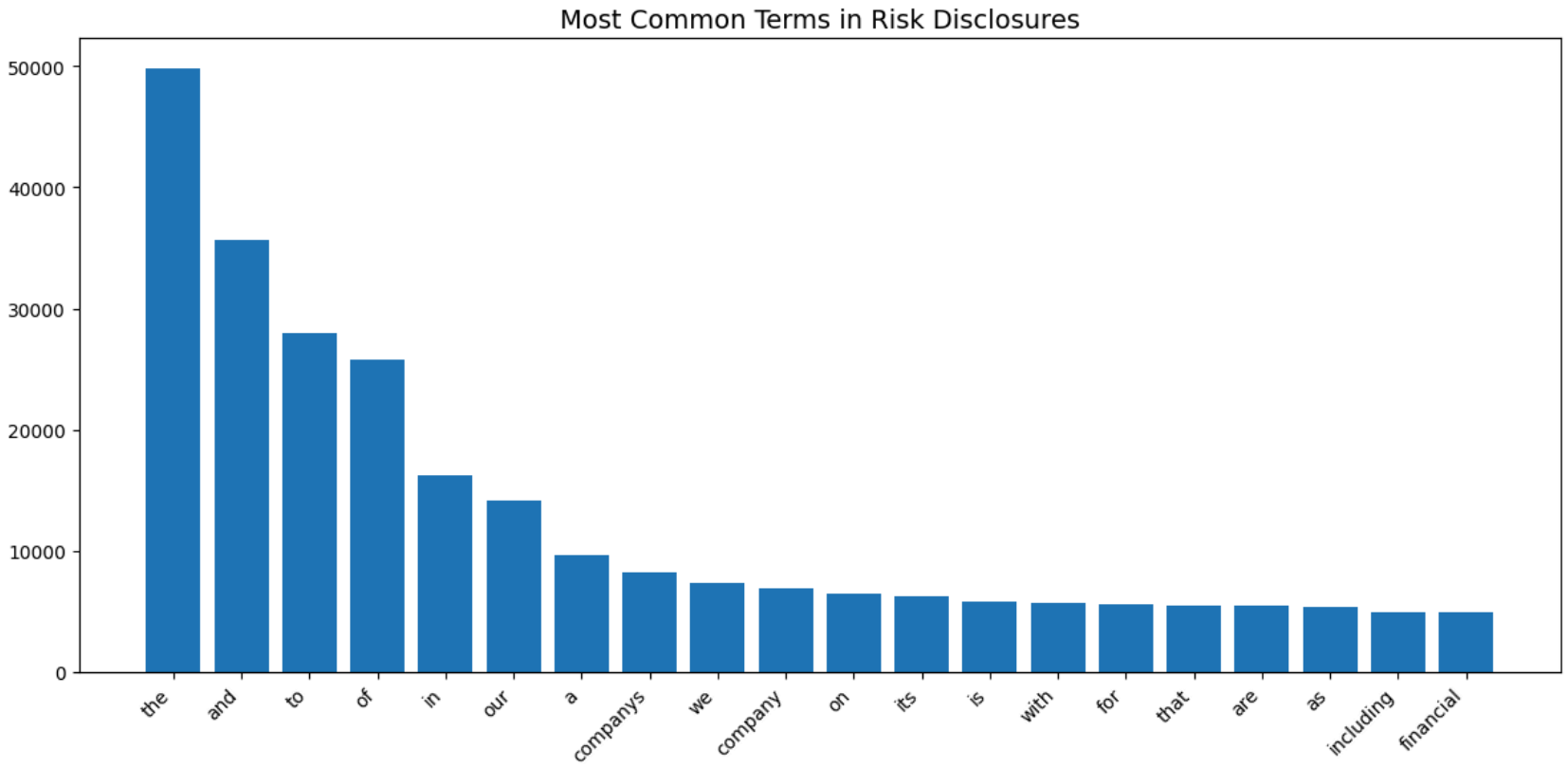
```
Text Length Statistics:
count      827.000000
mean      5753.576784
std       1307.528456
min       2421.000000
25%       4932.000000
50%       5622.000000
75%       6380.500000
max      12013.000000
Name: text_length, dtype: float64
```

```
# Text Length Analysis per severity (Box plot)
plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x='risk_severity', y='text_length', order=severity_order, palette='magma', legend=False, hue='risk_severity')
plt.title('Text Length Distribution by Severity', fontsize=14)
plt.xlabel('Severity', fontsize=12)
plt.ylabel('Text Length (Number of Words)', fontsize=12)
plt.show()
```

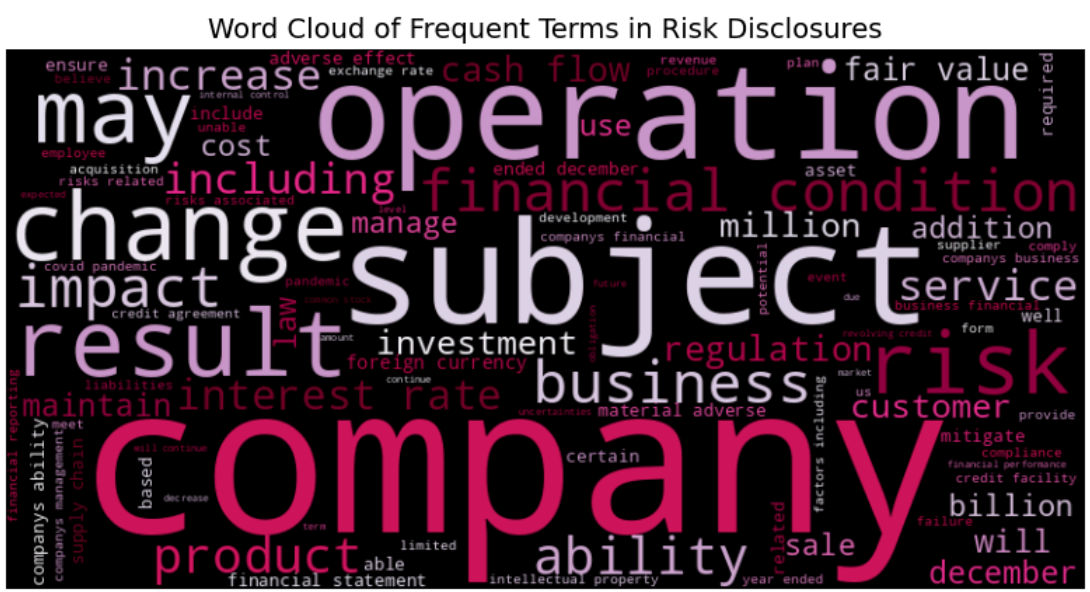


Texts range from ~2,400 to 12,000 characters with most around 5,700 Low severity texts tend to be shorter than others Normalizing text length or adding length-based features could be considered for modelling

```
# Define a simple tokenization function (issues with nltk)
def simple_preprocess(text):
    text = text.lower() # Convert to lowercase
    text = re.sub(r'[^a-zA-Z\s]', '', text) # Remove special characters and numbers
    text = re.sub(r'\s+', ' ', text).strip() # Remove extra whitespace
    return text


# Bar Chart with most common words
# Preprocess text and split into words
all_words = []
for text in df['input']:
    words = simple_preprocess(text).split()
    all_words.extend(words)

# Count word frequencies
word_counts = Counter(all_words)

common_words = word_counts.most_common(20)  # Top 20 words
words, counts = zip(*common_words)

plt.figure(figsize=(12, 6))
plt.bar(words, counts)
plt.xticks(rotation=45, ha='right')
plt.title('Most Common Terms in Risk Disclosures', fontsize=14)
plt.tight_layout()
plt.show()
```




```
# WordCloud
# Preprocess and join all text
all_text = ' '.join(df['input'].apply(simple_preprocess))

# Generate wordcloud
wordcloud = WordCloud(
    width=800,
    height=400,
    background_color='black',
    max_words=100,
    colormap='PuRd_r'
).generate(all_text)

# Display the wordcloud
plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Word Cloud of Frequent Terms in Risk Disclosures', fontsize=14)
plt.show()
```

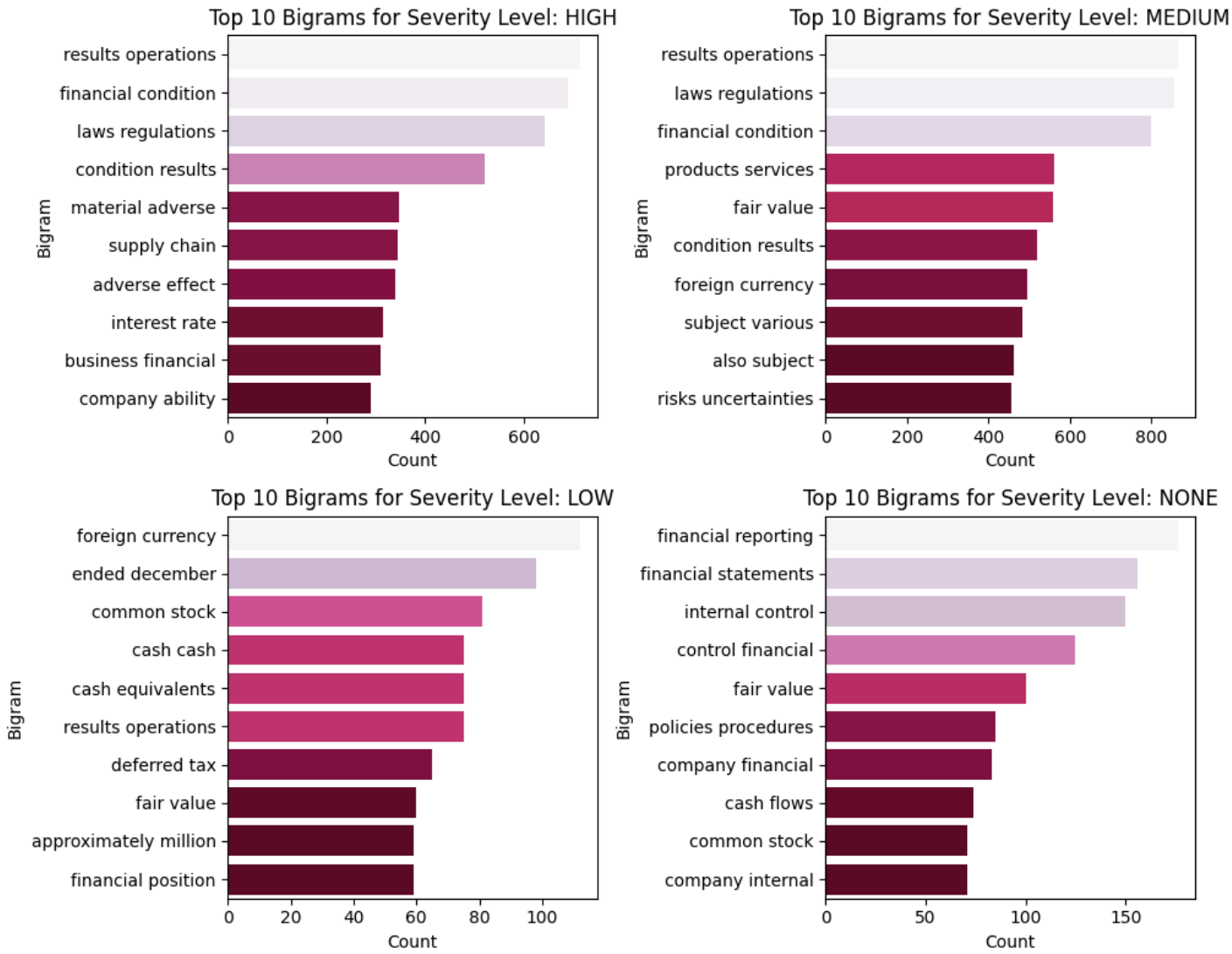



```
# WordCloud for each level of severity
# Create wordclouds for each severity level with different color schemes
plt.figure(figsize=(10, 8))

# Color schemes for each severity level
colormap_by_severity = {
    'HIGH': 'PuRd_r',
    'MEDIUM': 'PuOr_r',
    'LOW': 'BuPu_r',
    'NONE': 'YlGn'
}

for i, severity in enumerate(['HIGH', 'MEDIUM', 'LOW', 'NONE'], 1):
    # Filter data by severity
    severity_texts = df[df['risk_severity'] == severity]['input']
    # Preprocess and join
    text = ' '.join(severity_texts.apply(simple_preprocess))
    # Generate wordcloud with severity-specific colormap
    wc = WordCloud(
        width=800,
        height=400,
        background_color='black',
        max_words=100,
        colormap=colormap_by_severity[severity]
    ).generate(text)

    plt.subplot(2, 2, i)
    plt.imshow(wc, interpolation='bilinear')
    plt.axis('off')
```

```
    plt.title(f'Frequent Terms in Risk Disclosures: {severity} ', fontsize=10)

plt.tight_layout()
plt.show()
```


```
# n-grams analysis
def get_top_ngrams(texts, n=2, top_k=5):
    stop_words = set(stopwords.words('english'))
    tokens = []
    for text in texts:
        try:
            # NLTK tokenizer
            token_list = word_tokenize(str(text).lower())
        except:
            # Fallback to simple tokenization in nltk fails again
            token_list = str(text).lower().split()
        # Filter tokens
        filtered_tokens = [token for token in token_list if token.isalpha() and token not in stop_words]
        tokens.append(filtered_tokens)
```

```
    # Generate n-grams
    ngrams = []
    for token_list in tokens:
        ngrams.extend([' '.join(token_list[i:i+n]) for i in range(len(token_list)-n+1)])
    ngram_counts = Counter(ngrams)
    return ngram_counts.most_common(top_k)


# Top bigrams for each severity
plt.figure(figsize=(10, 8))
for i, severity in enumerate(severity_order, 1):
    plt.subplot(2, 2, i)
    severity_texts = df[df['risk_severity'] == severity]['input']
    top_bigrams = get_top_ngrams(severity_texts, n=2, top_k=10)
    bigram_df = pd.DataFrame(top_bigrams, columns=['bigram', 'count'])
    sns.barplot(data=bigram_df, x='count', y='bigram', palette='PuRd_r', hue='count', legend=False)
    plt.title(f'Top 10 Bigrams for Severity Level: {severity}', fontsize=12)
    plt.xlabel('Count', fontsize=10)
    plt.ylabel('Bigram', fontsize=10)
plt.tight_layout()
plt.show()
```




```
# Statistics for top bigrams
print("\nTop 10 Bigrams by Severity:")
for severity in severity_order:
    severity_texts = df[df['risk_severity'] == severity]['input']
    top_bigrams = get_top_ngrams(severity_texts, n=2, top_k=10)
    print(f"\n{severity}:")
    print(top_bigrams)
```

```
Top 10 Bigrams by Severity:
```

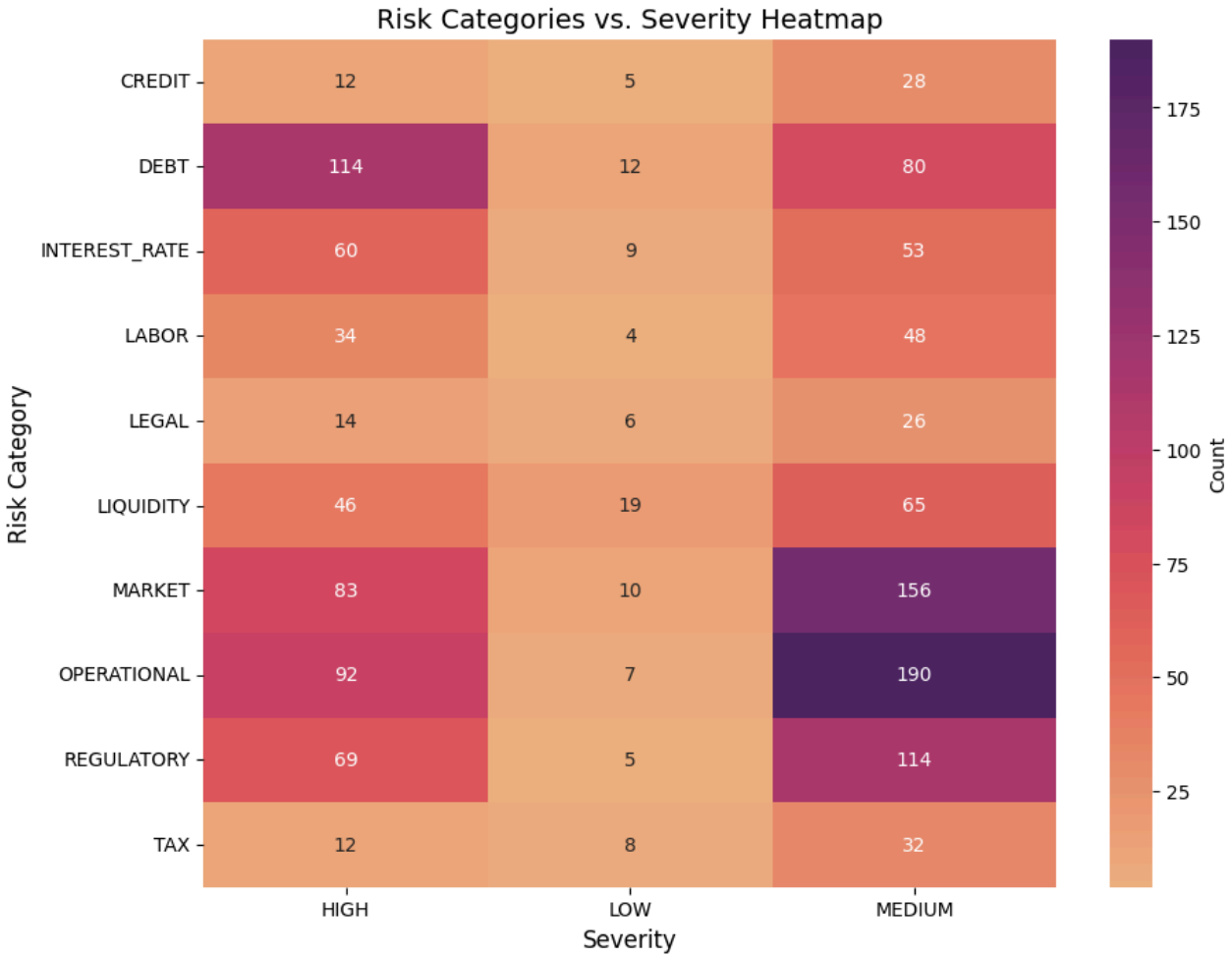
```
HIGH:
[('results operations', 714), ('financial condition', 691), ('laws regulations', 642), ('condition results', 520), ('material adverse', 347), ('supply chain', 344), ('adverse effect', 34

MEDIUM:
[('results operations', 864), ('laws regulations', 856), ('financial condition', 799), ('products services', 562), ('fair value', 558), ('condition results', 518), ('foreign currency', 4

LOW:
[('foreign currency', 112), ('ended december', 98), ('common stock', 81), ('cash cash', 75), ('cash equivalents', 75), ('results operations', 75), ('deferred tax', 65), ('fair value', 60

NONE:
[('financial reporting', 176), ('financial statements', 156), ('internal control', 150), ('control financial', 125), ('fair value', 100), ('policies procedures', 85), ('company financial
```

```
# Statistics for top trigrams
print("\nTop 10 Trigrams by Severity:")
for severity in severity_order:
    severity_texts = df[df['risk_severity'] == severity]['input']
    top_bigrams = get_top_ngrams(severity_texts, n=3, top_k=10)
    print(f"\n{severity}:")
    print(top_bigrams)
```

```
Top 10 Trigrams by Severity:

HIGH:
[('financial condition results', 520), ('condition results operations', 519), ('material adverse effect', 314), ('business financial condition', 254), ('could material adverse', 251), ('

MEDIUM:
[('financial condition results', 518), ('condition results operations', 517), ('year ended december', 309), ('business financial condition', 306), ('material adverse effect', 290), ('cou

LOW:
[('cash cash equivalents', 75), ('year ended december', 57), ('deferred tax assets', 46), ('financial position results', 46), ('position results operations', 46), ('foreign currency exch

NONE:
[('internal control financial', 125), ('control financial reporting', 125), ('company internal control', 61), ('current report form', 48), ('year ended december', 47), ('property plant e
```

```
# Heatmap Risk Severity and Risk Category
severity_risk_cat = df.explode('risk_categories')
pivot_table = pd.pivot_table(
    severity_risk_cat,
    index='risk_categories',
    columns='risk_severity',
    values='input',
    aggfunc='count',
    fill_value=0
)

# Heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(pivot_table, annot=True, fmt='d', cmap='flare', cbar_kws={'label': 'Count'})
plt.title('Risk Categories vs. Severity Heatmap', fontsize=14)
plt.xlabel('Severity', fontsize=12)
```

```
plt.ylabel('Risk Category', fontsize=12)
plt.show()
```



Strong correlation between certain categories and severity levels (visible in heatmap) Risk categories could be valuable features for classification

Text preprocessing

The text processing from this section is for ML algorithms

Different steps would be taken for more advance models such as Transformers etc

```
# Based on what is observed during EDA and domain knowledge, we define a set of important financial words to keep after stopwords are removed
financial_terms_to_keep = [
    'debt', 'risk', 'interest', 'rate', 'market', 'credit', 'liability',
    'liabilities', 'asset', 'assets', 'financial', 'liquidity', 'regulatory',
    'compliance', 'material', 'adverse', 'effect', 'exposure', 'increase',
    'decrease', 'volatility', 'covenant', 'default', 'obligation', 'tax',
    'capital', 'cash', 'flow', 'operational', 'operation', 'legal'
]


# Get standard English stopwords but remove financial terms we want to keep
stop_words = set(stopwords.words('english'))
stop_words = {word for word in stop_words if word not in financial_terms_to_keep}


# We also remove common stopwords that are usuually present only in financial documents
common_stopwords = {
    'company', 'companies', 'corporation', 'inc', 'incorporated', 'llc',
    'item', 'pursuant', 'thereof', 'thereto', 'therein', 'including',
    'include', 'includes', 'may', 'could', 'would', 'regarding'
}
stop_words.update(common_stopwords)


# We are using Lemmantization to reduce vocabulary size (by converting workd to their base form) which
# may help reduce feature dimensionality and help with generalization
lemmatizer = WordNetLemmatizer()


# We define the function below to address issues found in EDA and in general for financial text
# This is also our text preprocessing pipeline for machine learning models
def preprocess_financial_text(text, lemmatize=True, remove_stopwords=True):
    # Convert to string and lowercase (for case insensitive because of ML algorithms)
    text = str(text).lower()

    # Remove quotes and other artifacts from the dataset
    text = text.replace('"', ' ').replace("'", ' ').replace('\\n', ' ')

    # Replace financial metrics patterns
    text = re.sub(r'\$\s*(\d+(?:\.\d+)?)\s*billion', r'dollar \1 billion', text)
    text = re.sub(r'\$\s*(\d+(?:\.\d+)?)\s*million', r'dollar \1 million', text)
    text = re.sub(r'\$\s*(\d+(?:\.\d+)?)\s*thousand', r'dollar \1 thousand', text)
    text = re.sub(r'(\d+(?:\.\d+)?)\s*%', r'\1 percent', text)
    text = re.sub(r'(\d{1,2})[/-](\d{1,2})[/-](\d{2,4})', r'date \1 \2 \3', text)

    # Tokenize
    tokens = word_tokenize(text)

    # Remove punctuation and non-alphabetic tokens except preserved terms
    cleaned_tokens = []
    for token in tokens:
        # Keep tokens with preserved financial terms
        if ('dollar' in token or 'percent' in token or 'billion' in token or
            'million' in token or 'date' in token):
            cleaned_tokens.append(token)
```

```
            # Keep alphabet tokens
            elif token.isalpha():
                cleaned_tokens.append(token)

        if remove_stopwords:
            cleaned_tokens = [token for token in cleaned_tokens if token not in stop_words]

        # Apply Lemmatization if needed
        if lemmatize:
            cleaned_tokens = [lemmatizer.lemmatize(token) for token in cleaned_tokens]

        # Output processed text
        return ' '.join(cleaned_tokens)


    # Full preprocessing (lemmatization + stopword removal)
    df['processed_full'] = df['input'].apply(
        lambda x: preprocess_financial_text(x, lemmatize=True, remove_stopwords=True)
    )
```

Once the corpus is tokenized, we check below the top 10 terms for eaach severity level.

```
# Function to extract top terms by severity level
def extract_top_terms_by_severity(df, column, n=10):
    top_terms = {}
    for severity in ['HIGH', 'MEDIUM', 'LOW', 'NONE']:
        # Get all texts for this severity
        texts = ' '.join(df[df['risk_severity'] == severity][column])
        # Count term frequencies
        words = texts.split()
        word_counts = Counter(words)
        # Get top terms
        top_terms[severity] = word_counts.most_common(n)
    return top_terms

# Extract top terms for each preprocessing approach
top_terms_full = extract_top_terms_by_severity(df, 'processed_full')


# Print top terms for each severity level
print("Top terms by severity level (with full preprocessing):")
for severity, terms in top_terms_full.items():
    print(f"\n{severity}:")
    for term, count in terms:
        print(f"  {term}: {count}")
```

```
Top terms by severity level (with full preprocessing):

 HIGH:
   risk: 1860
   financial: 1626
   business: 1531
   result: 1463
```

```
    operation: 1336
    impact: 1266
    dollar: 1109
    subject: 1107
    also: 1086
    condition: 1009

  MEDIUM:
    risk: 3212
    financial: 2496
    business: 2277
    operation: 1998
    dollar: 1879
    result: 1879
    subject: 1838
    also: 1790
    tax: 1723
    product: 1688

  LOW:
    dollar: 399
    tax: 386
    financial: 283
    million: 278
    cash: 263
    risk: 262
    rate: 232
    december: 227
    share: 222
    asset: 205

  NONE:
    financial: 574
    statement: 259
    control: 228
    reporting: 219
    plan: 211
    agreement: 211
    stock: 211
    asset: 209
    value: 208
    investment: 199
```

Feature Engineering

From EDA observations, we believe those features may be useful in analysis but those features may not necessarily be used in modelling as they may not be available readily when implementing the pipeline in a real world situation

```
# From EDA, we have seen that Text length varies depending on severity level.
# Low severity tends to be shorter than high and medium level of severity
df['text_length'] = df['input'].apply(len)

# Number of tokens help calculate the length of the document after preprocessing
```

```
# High and medium severity risk tend to be longer
df['num_tokens'] = df['processed_full'].apply(lambda x: len(x.split()))


# We use the list of financial terms created earlier (more of these terms may be mentioned for higher severity)
# From EDA (Wordcloud), we have seen key financial terms and bigram analysis showed certain financial specififc phrases
df['num_financial_terms'] = df['processed_full'].apply(
    lambda x: sum(1 for token in x.split() if token in financial_terms_to_keep)
)
```

Target Variable

```
# Encoding
severity_mapping = {
    'NONE': 0,
    'LOW': 1,
    'MEDIUM': 2,
    'HIGH': 3
}

# Apply the mapping to create a new target variable
df['severity'] = df['risk_severity'].map(severity_mapping)
test_df['severity'] = test_df['risk_severity'].map(severity_mapping)


# Verify the mapping
print(df[['risk_severity', 'severity']].head())
```

```
  risk_severity  severity
0          HIGH         3
1          HIGH         3
2        MEDIUM         2
3          HIGH         3
4           LOW         1
```

```
# For base/ml models
# df['processed_full'] is used as input for feature extractors (applied the necessary 'transformations')
```

Modelling

To create our pipeline to analyse financial risk severity from annual financial report; we initially was using an 80:20 Train Validation Split; despite small dataset. We opted for this due to hardware restriction as we believed that Kfold cross validation would be computationally expensive when implemented on Transformer Models.

However, after observing poor performance (overfitting, poor prediction on minority classes); we decided to implement Kfold as it would be more ideal for our dataset. The results from previous train:val analysis is availabe in a seperate notebook and can be provided upon request.

Base Models

Logistic Regression (LR) and Support Vector Machine (SVM)

We will be exploring TF-IDF and BoW with Bi-grams for our base models. We are opting for Uni-grams and Bi-grams as those were more significant as observed in EDA. Tri-grams showed less distinctive patterns across severity levels compared to bigrams. We observed overlapping and trigrams are not that significant and may cause sparsity. Bigrams showed distinctive patterns among the severity levels; for example, "supply chain" appears prominently in high severity but not in others, which could be a useful signal.

```
# Import necessaary libraries
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import classification_report, accuracy_score, make_scorer, f1_score
from sklearn.model_selection import GridSearchCV
from imblearn.over_sampling import SMOTE
```

```
# Texts and labels used for base models/machine learning models
# Full preprocessing (lemmatization + stopword removal)
df['processed_full'] = df['input'].apply(
    lambda x: preprocess_financial_text(x, lemmatize=True, remove_stopwords=True))

y_train = df['severity']
```

```
# We apply BoW and TF-IDF on previously processed texts (i.e stopwords removal and lemmatization)
# We limit number of features to 5000 for efficiency and avoid overfitting
# We exclude terms that appear in less than 5 documents
# Bag of Words (BoW)
bow_vectorizer = CountVectorizer(ngram_range=(1, 2), max_features=5000, min_df=5)
X_train_bow = bow_vectorizer.fit_transform(df['processed_full'])

# TF-IDF
tfidf_vectorizer = TfidfVectorizer(ngram_range=(1, 2), max_features=5000, min_df=5)
X_train_tfidf = tfidf_vectorizer.fit_transform(df['processed_full'])
```

```
# Kfold cross validation initialisation
# We use 'stratified' to ensure a balanced class distribution across folds
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

Logistic Regression (LR)

```
# Logistic Regression (BoW)
baselr_bow = LogisticRegression(class_weight='balanced', random_state=42) # using class weights as balanced since dataset was imbalanced
# Perform 5-fold Cross-Validation
baselrbow_accuracy = cross_val_score(baselr_bow, X_train_bow, y_train, cv=cv, scoring='accuracy')
baselrbow_f1 = cross_val_score(baselr_bow, X_train_bow, y_train, cv=cv, scoring='f1_macro')
```

```
print("Bag of Words with LR Results:")
print(f"Accuracy: {baselrbow_accuracy.mean():.4f} ± {baselrbow_accuracy.std():.4f}")
print(f"F1 Score: {baselrbow_f1.mean():.4f} ± {baselrbow_f1.std():.4f}")

# Logistic Regression (TF-IDF)
baselr_tfidf = LogisticRegression(class_weight='balanced', random_state=42)
# Perform 5-fold Cross-Validation
baselrtfidf_accuracy = cross_val_score(baselr_tfidf, X_train_tfidf, y_train, cv=cv, scoring='accuracy')
baselrtfidf_f1 = cross_val_score(baselr_tfidf, X_train_tfidf, y_train, cv=cv, scoring='f1_macro')
print("\nTF-IDF with LR Results:")
print(f"Accuracy: {baselrtfidf_accuracy.mean():.4f} ± {baselrtfidf_accuracy.std():.4f}")
print(f"F1 Score: {baselrtfidf_f1.mean():.4f} ± {baselrtfidf_f1.std():.4f}")
```

```
Bag of Words with LR Results:
Accuracy: 0.6046 ± 0.0141
F1 Score: 0.4828 ± 0.0258

TF-IDF with LR Results:
Accuracy: 0.5767 ± 0.0269
F1 Score: 0.4990 ± 0.0156
```

Support Vector Machine (SVM/SVC)

```
# SVM (BoW)
basesvm_bow = SVC(class_weight='balanced', random_state=42)
basesvmbow_accuracy = cross_val_score(basesvm_bow, X_train_bow, y_train, cv=cv, scoring='accuracy')
basesvmbow_f1 = cross_val_score(basesvm_bow, X_train_bow, y_train, cv=cv, scoring='f1_macro')
print("Bag of Words with SVM Results:")
print(f"Accuracy: {basesvmbow_accuracy.mean():.4f} ± {basesvmbow_accuracy.std():.4f}")
print(f"F1 Score: {basesvmbow_f1.mean():.4f} ± {basesvmbow_f1.std():.4f}")

# SVM (TF-IDF)
basesvm_tfidf = SVC(class_weight='balanced', random_state=42)
basesvmtfidf_accuracy = cross_val_score(basesvm_tfidf, X_train_tfidf, y_train, cv=cv, scoring='accuracy')
basesvmtfidf_f1 = cross_val_score(basesvm_tfidf, X_train_tfidf, y_train, cv=cv, scoring='f1_macro')
print("\nTF-IDF with SVM Results:")
print(f"Accuracy: {basesvmtfidf_accuracy.mean():.4f} ± {basesvmtfidf_accuracy.std():.4f}")
print(f"F1 Score: {basesvmtfidf_f1.mean():.4f} ± {basesvmtfidf_f1.std():.4f}")
```

```
Bag of Words with SVM Results:
Accuracy: 0.6166 ± 0.0219
F1 Score: 0.5333 ± 0.0195

TF-IDF with SVM Results:
Accuracy: 0.6287 ± 0.0171
F1 Score: 0.4630 ± 0.0253
```

For LR, BoW slightly outperforms TF-IDF. Standard deviations are also relatively small (suggestin stability). However, looking at the difference between accuracy and F1, we can conclude that the model may be struggling with minority classes. We also suspect overfitting (we will fit the model to training set to see if there is overfitting).

For SVM, TF-IDF outperforms BoW. There is stability during training (Low STD). We can see quite a big difference with F1 score and accuracy again. We need to check for overfitting (we suspect it since the dataset is quite small)

To attempt improving generalizability, we will reducfe features to 3000.

```
# Quick overfitting check
baselr_bow = LogisticRegression(class_weight='balanced', random_state=42)
baselr_bow.fit(X_train_bow, y_train)
train_accuracy = baselr_bow.score(X_train_bow, y_train)
print(f"LR Training accuracy: {train_accuracy:.4f}")
print(f"LR Cross-val accuracy: {baselrbow_accuracy.mean():.4f}")

basesvm_bow = SVC(class_weight='balanced', random_state=42)
basesvm_bow.fit(X_train_bow, y_train)
train_accuracy = basesvm_bow.score(X_train_bow, y_train)
print(f"\nSVM Training accuracy: {train_accuracy:.4f}")
print(f"SVM Cross-val accuracy: {basesvmbow_accuracy.mean():.4f}")
```

```
LR Training accuracy: 1.0000
LR Cross-val accuracy: 0.6046

SVM Training accuracy: 0.8356
SVM Cross-val accuracy: 0.6166
```

As suspected, we can observe overfitting. Severe overfitting in both models (slightly lower in SVW).

Below we try to reduce maximum number of features to see if we could reduce overfitting.

```
# Bag of Words (BoW)
bow_vectorizer = CountVectorizer(ngram_range=(1, 2), max_features=3000, min_df=5)
X_train_bow = bow_vectorizer.fit_transform(df['processed_full'])

# TF-IDF
tfidf_vectorizer = TfidfVectorizer(ngram_range=(1, 2), max_features=3000, min_df=5)
X_train_tfidf = tfidf_vectorizer.fit_transform(df['processed_full'])

# Logistic Regression (BoW)
baselr_bow = LogisticRegression(class_weight='balanced', random_state=42) # setting class weights as 'balanced' since dataset was imbalanced
# Perform 5-fold Cross-Validation
baselrbow_accuracy = cross_val_score(baselr_bow, X_train_bow, y_train, cv=cv, scoring='accuracy')
baselrbow_f1 = cross_val_score(baselr_bow, X_train_bow, y_train, cv=cv, scoring='f1_macro')
print("Bag of Words with LR Results:")
print(f"Accuracy: {baselrbow_accuracy.mean():.4f} ± {baselrbow_accuracy.std():.4f}")
print(f"F1 Score: {baselrbow_f1.mean():.4f} ± {baselrbow_f1.std():.4f}")

# Logistic Regression (TF-IDF)
baselr_tfidf = LogisticRegression(class_weight='balanced', random_state=42)
# Perform 5-fold Cross-Validation
baselrtfidf_accuracy = cross_val_score(baselr_tfidf, X_train_tfidf, y_train, cv=cv, scoring='accuracy')
baselrtfidf_f1 = cross_val_score(baselr_tfidf, X_train_tfidf, y_train, cv=cv, scoring='f1_macro')
print("\nTF-IDF with LR Results:")
print(f"Accuracy: {baselrtfidf_accuracy.mean():.4f} ± {baselrtfidf_accuracy.std():.4f}")
```

```
print(f"F1 Score: {baselrtfidf_f1.mean():.4f} ± {baselrtfidf_f1.std():.4f}")

# SVM (BoW)
basesvm_bow = SVC(class_weight='balanced', random_state=42)
basesvmbow_accuracy = cross_val_score(basesvm_bow, X_train_bow, y_train, cv=cv, scoring='accuracy')
basesvmbow_f1 = cross_val_score(basesvm_bow, X_train_bow, y_train, cv=cv, scoring='f1_macro')
print("\nBag of Words with SVM Results:")
print(f"Accuracy: {basesvmbow_accuracy.mean():.4f} ± {basesvmbow_accuracy.std():.4f}")
print(f"F1 Score: {basesvmbow_f1.mean():.4f} ± {basesvmbow_f1.std():.4f}")

# SVM (TF-IDF)
basesvm_tfidf = SVC(class_weight='balanced', random_state=42)
basesvmtfidf_accuracy = cross_val_score(basesvm_tfidf, X_train_tfidf, y_train, cv=cv, scoring='accuracy')
basesvmtfidf_f1 = cross_val_score(basesvm_tfidf, X_train_tfidf, y_train, cv=cv, scoring='f1_macro')
print("\nTF-IDF with SVM Results:")
print(f"Accuracy: {basesvmtfidf_accuracy.mean():.4f} ± {basesvmtfidf_accuracy.std():.4f}")
print(f"F1 Score: {basesvmtfidf_f1.mean():.4f} ± {basesvmtfidf_f1.std():.4f}")
```

```
Bag of Words with LR Results:
Accuracy: 0.5816 ± 0.0320
F1 Score: 0.4652 ± 0.0401

TF-IDF with LR Results:
Accuracy: 0.5743 ± 0.0269
F1 Score: 0.4991 ± 0.0140

Bag of Words with SVM Results:
Accuracy: 0.6154 ± 0.0199
F1 Score: 0.5321 ± 0.0189

TF-IDF with SVM Results:
Accuracy: 0.6263 ± 0.0228
F1 Score: 0.4712 ± 0.0218
```

```
# Quick overfitting check
baselr_bow = LogisticRegression(class_weight='balanced', random_state=42)
baselr_bow.fit(X_train_bow, y_train)
train_accuracy = baselr_bow.score(X_train_bow, y_train)
print(f"LR Training accuracy: {train_accuracy:.4f}")
print(f"LR Cross-val accuracy: {baselrbow_accuracy.mean():.4f}")

basesvm_bow = SVC(class_weight='balanced', random_state=42)
basesvm_bow.fit(X_train_bow, y_train)
train_accuracy = basesvm_bow.score(X_train_bow, y_train)
print(f"\nSVM Training accuracy: {train_accuracy:.4f}")
print(f"SVM Cross-val accuracy: {basesvmbow_accuracy.mean():.4f}")
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

```
LR Training accuracy: 1.0000
LR Cross-val accuracy: 0.5816

SVM Training accuracy: 0.8295
SVM Cross-val accuracy: 0.6154
```

Reducing the number of features has not improve generalizability of the models. We will stick to 5000 features for the base models when optimising. We also experience convergence issue for Logistic regression.

Text Augmentation

As observed during EDA, 'NONE' and 'LOW' are minority classes. We will use Synonym Replacement Augmentation to oversample the minority classes which we would test during modelling to see if performance improves. The code to generate for text augmentation was from this article: https://neptune.ai/blog/data-augmentation-nlp where they also suggest using 'nlpaug', which we have attempted but this lead to a pipeline that is computationally expensive. We have instead opted for a more simple one.

Shahul ES. (2023, September 1). Data augmentation in NLP: Best practices from a Kaggle master. Neptune.ai. https://neptune.ai/blog/data-augmentation-nlp

```
from nltk.corpus import wordnet
from tqdm import tqdm
import random
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
True
```

```
def augment_text(df, samples=100, pr=0.2):
    new_text = []
    # Selecting the minority class samples
    df_n = df[df.severity == 0].reset_index(drop=True)
    # Data augmentation loop
    for i in tqdm(np.random.randint(0, len(df_n), samples)):
        text = df_n.iloc[i]['input']
        augmented_text = synonym_replacement(text, pr)
        new_text.append(augmented_text)
    # Create DataFrame for class 0
    new0 = pd.DataFrame({'input': new_text, 'severity': 0})

    # Reset new_text for class 1
    new_text = []
    # Selecting class 1 samples
    df_n = df[df.severity == 1].reset_index(drop=True)
```

```
    # Data augmentation loop for class 1
    for i in tqdm(np.random.randint(0, len(df_n), samples)):
        text = df_n.iloc[i]['input']
        augmented_text = synonym_replacement(text, pr)
        new_text.append(augmented_text)
    # Create DataFrame for class 1
    new1 = pd.DataFrame({'input': new_text, 'severity': 1})
    # Combine all data and shuffle
    from sklearn.utils import shuffle
    df = shuffle(pd.concat([df, new0, new1], ignore_index=True))
    return df

def synonym_replacement(text, percent=0.2):
    words = text.split()
    n_to_replace = max(1, int(len(words) * percent))
    indices = random.sample(range(len(words)), min(n_to_replace, len(words)))
    for idx in indices:
        if words[idx].isalpha() and len(words[idx]) > 3:  # Only replace actual words
            synonyms = []
            for syn in wordnet.synsets(words[idx]):
                for lemma in syn.lemmas():
                    synonyms.append(lemma.name())
            if synonyms:
                words[idx] = random.choice(synonyms)
    return ' '.join(words)

df = augment_text(df)
```

```
100%|██████████| 100/100 [00:00<00:00, 119.04it/s]
100%|██████████| 100/100 [00:00<00:00, 119.69it/s]
```

```
# Full preprocessing (lemmatization + stopword removal) of all texts including new data generated through synonym replacement
df['aug_processed_full'] = df['input'].apply(
    lambda x: preprocess_financial_text(x, lemmatize=True, remove_stopwords=True)
)

aug_y_train = df['severity']


# Bag of Words (BoW)
bow_vectorizer = CountVectorizer(ngram_range=(1, 2), max_features=5000, min_df=5)
X_train_bow = bow_vectorizer.fit_transform(df['aug_processed_full'])

# TF-IDF
tfidf_vectorizer = TfidfVectorizer(ngram_range=(1, 2), max_features=5000, min_df=5)
X_train_tfidf = tfidf_vectorizer.fit_transform(df['aug_processed_full'])

# Logistic Regression (BoW)
baselr_bow = LogisticRegression(class_weight='balanced', random_state=42) # using class weights as balanced since dataset was imbalanced
# Perform 5-fold Cross-Validation
baselrbow_accuracy = cross_val_score(baselr_bow, X_train_bow, aug_y_train, cv=cv, scoring='accuracy')
baselrbow_f1 = cross_val_score(baselr_bow, X_train_bow, aug_y_train, cv=cv, scoring='f1_macro')
print("Bag of Words with LR Results:")
```

```
print(f"Accuracy: {baselrbow_accuracy.mean():.4f} ± {baselrbow_accuracy.std():.4f}")
print(f"F1 Score: {baselrbow_f1.mean():.4f} ± {baselrbow_f1.std():.4f}")

# Logistic Regression (TF-IDF)
baselr_tfidf = LogisticRegression(class_weight='balanced', random_state=42)
# Perform 5-fold Cross-Validation
baselrtfidf_accuracy = cross_val_score(baselr_tfidf, X_train_tfidf, aug_y_train, cv=cv, scoring='accuracy')
baselrtfidf_f1 = cross_val_score(baselr_tfidf, X_train_tfidf, aug_y_train, cv=cv, scoring='f1_macro')
print("\nTF-IDF with LR Results:")
print(f"Accuracy: {baselrtfidf_accuracy.mean():.4f} ± {baselrtfidf_accuracy.std():.4f}")
print(f"F1 Score: {baselrtfidf_f1.mean():.4f} ± {baselrtfidf_f1.std():.4f}")

# SVM (BoW)
basesvm_bow = SVC(class_weight='balanced', random_state=42)
basesvmbow_accuracy = cross_val_score(basesvm_bow, X_train_bow, aug_y_train, cv=cv, scoring='accuracy')
basesvmbow_f1 = cross_val_score(basesvm_bow, X_train_bow, aug_y_train, cv=cv, scoring='f1_macro')
print("\nBag of Words with SVM Results:")
print(f"Accuracy: {basesvmbow_accuracy.mean():.4f} ± {basesvmbow_accuracy.std():.4f}")
print(f"F1 Score: {basesvmbow_f1.mean():.4f} ± {basesvmbow_f1.std():.4f}")

# SVM (TF-IDF)
basesvm_tfidf = SVC(class_weight='balanced', random_state=42)
basesvmtfidf_accuracy = cross_val_score(basesvm_tfidf, X_train_tfidf, aug_y_train, cv=cv, scoring='accuracy')
basesvmtfidf_f1 = cross_val_score(basesvm_tfidf, X_train_tfidf, aug_y_train, cv=cv, scoring='f1_macro')
print("\nTF-IDF with SVM Results:")
print(f"Accuracy: {basesvmtfidf_accuracy.mean():.4f} ± {basesvmtfidf_accuracy.std():.4f}")
print(f"F1 Score: {basesvmtfidf_f1.mean():.4f} ± {basesvmtfidf_f1.std():.4f}")
```

```
Bag of Words with LR Results:
Accuracy: 0.7138 ± 0.0267
F1 Score: 0.7424 ± 0.0257

TF-IDF with LR Results:
Accuracy: 0.6943 ± 0.0292
F1 Score: 0.7074 ± 0.0298

Bag of Words with SVM Results:
Accuracy: 0.7215 ± 0.0239
F1 Score: 0.7394 ± 0.0278

TF-IDF with SVM Results:
Accuracy: 0.7527 ± 0.0300
F1 Score: 0.7713 ± 0.0300
```

```
# Quick overfitting check
baselr_bow = LogisticRegression(class_weight='balanced', random_state=42)
baselr_bow.fit(X_train_bow, aug_y_train)
train_accuracy = baselr_bow.score(X_train_bow, aug_y_train)
print(f"LR(BoW) Training accuracy: {train_accuracy:.4f}")
print(f"LR(BoW) Cross-val accuracy: {baselrbow_accuracy.mean():.4f}")

basesvm_tfidf = SVC(class_weight='balanced', random_state=42)
basesvm_tfidf.fit(X_train_tfidf, aug_y_train)
train_accuracy = basesvm_tfidf.score(X_train_tfidf, aug_y_train)
```

```
print(f"\nSVM(TFIDF) Training accuracy: {train_accuracy:.4f}")
print(f"SVM(TFIDF) Cross-val accuracy: {basesvmtfidf_accuracy.mean():.4f}")
```

```
LR(BoW) Training accuracy: 1.0000
LR(BoW) Cross-val accuracy: 0.7215

SVM(TFIDF) Training accuracy: 0.9494
SVM(TFIDF) Cross-val accuracy: 0.7565
```

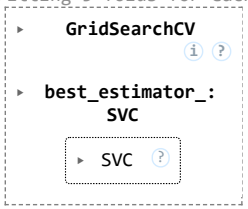
Using the additional sample has improved performance of the models (great improvement in F1 score) however overfitting still persists (same as before)

Grid Search

We optimise TF-IDF+SVM first, to attempt on improving the model and reduce overfitting. We might explore BoW+LR.

```
# Parameter grid
param_grid = {
    'C': [0.01, 0.1, 1, 10, 100],  # Regularization strength (Higher values included to mitigate overfitting)
    'class_weight': ['balanced', None],
    'kernel': ['linear', 'rbf'],  # Include RBF to see if non-linear helps
    'gamma': [0.1, 0.01]  # Only relevant for non-linear kernels
}

scoring = {
    'accuracy': make_scorer(accuracy_score),
    'f1_macro': make_scorer(f1_score, average='macro')
}

# Grid search with cross-validation
grid_search = GridSearchCV(
    SVC(random_state=42),
    param_grid,
    cv=5,
    scoring=scoring,
    refit='f1_macro',  # Refit on the best
    verbose=1,
    n_jobs=-1
)

# Fit on augmented data
grid_search.fit(X_train_tfidf, aug_y_train)
```

```
Fitting 5 folds for each of 40 candidates, totalling 200 fits
```




```
# Get best parameters
print(f"Best parameters: {grid_search.best_params_}")
print(f"Best F1 score: {grid_search.best_score_:.4f}")
print(f"Accuracy with best model: {grid_search.cv_results_['mean_test_accuracy'][grid_search.best_index_]:.4f}")
```

```
Best parameters: {'C': 10, 'class_weight': None, 'gamma': 0.1, 'kernel': 'rbf'}
Best F1 score: 0.7654
Accuracy with best model: 0.7469
```

```
# Get best model from grid search
best_model = grid_search.best_estimator_

# Check training performance
train_accuracy = best_model.score(X_train_tfidf, aug_y_train)
train_preds = best_model.predict(X_train_tfidf)
train_f1 = f1_score(aug_y_train, train_preds, average='macro')

# Get cross-validation performance
cv_accuracy = grid_search.cv_results_['mean_test_accuracy'][grid_search.best_index_]
cv_f1 = grid_search.best_score_  # This is already the F1 score

print(f"Grid Search SVM Training accuracy: {train_accuracy:.4f}")
print(f"Grid Search SVM Cross-val accuracy: {cv_accuracy:.4f}")
print(f"Grid Search SVM Training F1: {train_f1:.4f}")
print(f"Grid Search SVM Cross-val F1: {cv_f1:.4f}")

# Calculate gaps
acc_gap = train_accuracy - cv_accuracy
f1_gap = train_f1 - cv_f1
print(f"Accuracy gap: {acc_gap:.4f}")
print(f"F1 gap: {f1_gap:.4f}")
```

```
Grid Search SVM Training accuracy: 0.9591
Grid Search SVM Cross-val accuracy: 0.7469
Grid Search SVM Training F1: 0.9616
Grid Search SVM Cross-val F1: 0.7654
Accuracy gap: 0.2122
F1 gap: 0.1961
```

The best model from Grid Search shows worst performance that the base model with augmented data. We will attempt to reduce overfitting on the base model by using a smalled value for C.

```
# SVM (TF-IDF)
basesvm_tfidf = SVC(C=0.1, class_weight='balanced', random_state=42)
basesvmtfidf_accuracy = cross_val_score(basesvm_tfidf, X_train_tfidf, aug_y_train, cv=cv, scoring='accuracy')
basesvmtfidf_f1 = cross_val_score(basesvm_tfidf, X_train_tfidf, aug_y_train, cv=cv, scoring='f1_macro')
print("TF-IDF with SVM Results:")
print(f"Accuracy: {basesvmtfidf_accuracy.mean():.4f} ± {basesvmtfidf_accuracy.std():.4f}")
print(f"F1 Score: {basesvmtfidf_f1.mean():.4f} ± {basesvmtfidf_f1.std():.4f}")

basesvm_tfidf = SVC(class_weight='balanced', random_state=42)
basesvm_tfidf.fit(X_train_tfidf, aug_y_train)
train_accuracy = basesvm_tfidf.score(X_train_tfidf, aug_y_train)
print(f"\nSVM(TFIDF) Training accuracy: {train_accuracy:.4f}")
print(f"SVM(TFIDF) Cross-val accuracy: {basesvmtfidf_accuracy.mean():.4f}")
```

```
TF-IDF with SVM Results:
Accuracy: 0.5315 ± 0.0587
F1 Score: 0.4735 ± 0.0396

SVM(TFIDF) Training accuracy: 0.9494
SVM(TFIDF) Cross-val accuracy: 0.5315
```

Lower C lead to decrease in performance where F1 score decreases drsastically while the model overfit severely. Best Model so far is BASE TF-IDF SVM with Text augmented data and no grid search.

Advanced models

We will implement and evaluate FinBERT for our financial risk severity classification task. We may explore RoBERTa to provide a good contrast as RoBERTa is not pretrained for finance unlike FinBERT. We expect minority class prediction should improve and these models may even capture better semantic relationships. However overfitting may persist since the dataset is quite small for a transformer model with a lot of parameters. We will attempt to mitigate overfitting through weight decay, early stopping, dropout or a combination of those.

We will be using the data with Text Augmentation as seen in during base model, the augmented data improved performance.

```
from transformers import logging
logging.set_verbosity_error()
```

```
The cache for model files in Transformers v4.22.0 has been updated. Migrating your old cache. This is a one-time only operation. You can interrupt this and resume the migration later on
0/0 [00:00<?, ?it/s]
```

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, Dataset
from transformers import AutoModelForSequenceClassification, AutoTokenizer
from sklearn.metrics import f1_score, classification_report, confusion_matrix
from sklearn.utils.class_weight import compute_class_weight
from sklearn.model_selection import KFold
```

```
from sklearn.metrics import accuracy_score
from tqdm import tqdm
import random
from nltk.corpus import wordnet


# Text augmentation pipeline
def augment_text(df, samples=100, pr=0.2):
    new_text = []
    # Selecting the minority class samples
    df_n = df[df.severity == 0].reset_index(drop=True)
    # Data augmentation loop
    for i in tqdm(np.random.randint(0, len(df_n), samples)):
        text = df_n.iloc[i]['input']
        augmented_text = synonym_replacement(text, pr)
        new_text.append(augmented_text)
    # Create DataFrame for class 0
    new0 = pd.DataFrame({'input': new_text, 'severity': 0})

    # Reset new_text for class 1
    new_text = []
    # Selecting class 1 samples
    df_n = df[df.severity == 1].reset_index(drop=True)
    # Data augmentation loop for class 1
    for i in tqdm(np.random.randint(0, len(df_n), samples)):
        text = df_n.iloc[i]['input']
        augmented_text = synonym_replacement(text, pr)
        new_text.append(augmented_text)
    # Create DataFrame for class 1
    new1 = pd.DataFrame({'input': new_text, 'severity': 1})
    # Combine all data and shuffle
    from sklearn.utils import shuffle
    df = shuffle(pd.concat([df, new0, new1], ignore_index=True))
    return df

def synonym_replacement(text, percent=0.2):
    words = text.split()
    n_to_replace = max(1, int(len(words) * percent))
    indices = random.sample(range(len(words)), min(n_to_replace, len(words)))
    for idx in indices:
        if words[idx].isalpha() and len(words[idx]) > 3:  # Only replace actual words
            synonyms = []
            for syn in wordnet.synsets(words[idx]):
                for lemma in syn.lemmas():
                    synonyms.append(lemma.name())
            if synonyms:
                words[idx] = random.choice(synonyms)
    return ' '.join(words)

df = augment_text(df)
```

```
100%|██████████| 100/100 [00:03<00:00, 26.16it/s]
100%|██████████| 100/100 [00:00<00:00, 385.48it/s]
```

```
# For transformer models
texts = df['input'].tolist()
labels = df['severity'].tolist()


device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(device) # Check if GPU is selected before running the functions
```

```
cuda
```

```
# Custom Dataset class for FinBERT/ RoBERTa
class FinancialRiskDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_length=512):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = str(self.texts[idx])
        label = self.labels[idx]
        encoding = self.tokenizer(
            text,
            truncation=True,
            padding='max_length',
            max_length=self.max_length,
            return_tensors='pt'
        )
        return {
            'input_ids': encoding['input_ids'].squeeze(),
            'attention_mask': encoding['attention_mask'].squeeze(),
            'labels': torch.tensor(label, dtype=torch.long)
        }
```

```
# Training loop
def train(model, train_loader, optimizer, loss_fn, device):
    model.train()
    total_loss = 0
    for batch in train_loader:
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)
        optimizer.zero_grad()
        outputs = model(input_ids=input_ids, attention_mask=attention_mask).logits
        loss = loss_fn(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    avg_loss = total_loss / len(train_loader)
    return avg_loss


# Evaluation loop
def evaluate(model, data_loader, loss_fn, device):
    model.eval()
    all_preds = []
    all_labels = []
    total_loss = 0
    with torch.no_grad():
        for batch in data_loader:
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            labels = batch['labels'].to(device)
            outputs = model(input_ids=input_ids, attention_mask=attention_mask).logits
            loss = loss_fn(outputs, labels)
            total_loss += loss.item()
            preds = outputs.argmax(dim=-1)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())
    avg_loss = total_loss / len(data_loader)
    macro_f1 = f1_score(all_labels, all_preds, average='macro')
    return avg_loss, macro_f1, all_preds, all_labels


# Main Training Function (Kfold cross validation)
# We have previously experimented with 3 epochs (hence we set it as the default), however we saw that F1_score was still
# improving, so the models will be run using 5 epochs.
def kfold_train(texts, labels, model_fn, optimizer_fn, loss_fn, tokenizer, folds=5, epochs=3, batch_size=8, device=None):
    if device is None:
        device = 'cuda' if torch.cuda.is_available() else 'cpu'

    kf = KFold(n_splits=folds, shuffle=True, random_state=42) # Split data for each fold
    fold_accuracies = []
    fold_f1_scores = []

    # Variables to track best model across all folds
    best_overall_f1 = 0
    best_model_state = None
```

```
    for fold, (train_idx, val_idx) in enumerate(kf.split(texts)):
        print(f"\nFold {fold+1}/{folds}")

        # Prepare fold data
        train_texts = [texts[i] for i in train_idx]
        train_labels = [labels[i] for i in train_idx]
        val_texts = [texts[i] for i in val_idx]
        val_true_labels = [labels[i] for i in val_idx]

        # Create datasets and dataloaders
        train_dataset = FinancialRiskDataset(train_texts, train_labels, tokenizer)
        val_dataset = FinancialRiskDataset(val_texts, val_true_labels, tokenizer)
        train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
        val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)

        # Instantiate model
        model = model_fn().to(device)
        # Optimizer initialisation
        optimizer = optimizer_fn(model)

        best_fold_f1 = 0
        best_fold_model = None

        # Training loop
        for epoch in range(epochs):
            train_loss = train(model, train_loader, optimizer, loss_fn, device)
            val_loss, val_f1, val_preds, _ = evaluate(model, val_loader, loss_fn, device)
            print(f"  Epoch {epoch+1}/{epochs} | Train Loss: {train_loss:.4f} | Val Loss: {val_loss:.4f} | Val F1: {val_f1:.4f}")

            # Tracking the best model per fold
            if val_f1 > best_fold_f1:
                best_fold_f1 = val_f1
                best_fold_model = model.state_dict().copy()

                # Best model across all folds
                if val_f1 > best_overall_f1:
                    best_overall_f1 = val_f1
                    best_model_state = model.state_dict().copy()

        # Load the best model for this fold
        model.load_state_dict(best_fold_model)

        # Evaluation using function above (using f1 score because of imbalanced dataset)
        val_loss, val_f1, val_preds, _ = evaluate(model, val_loader, loss_fn, device)
        val_accuracy = accuracy_score(val_true_labels, val_preds)
        # Tracking fold results
        fold_accuracies.append(val_accuracy)
        fold_f1_scores.append(best_fold_f1) # save the model with best f1 rather than final epoch
        print(f"  Fold Accuracy: {val_accuracy:.4f}, F1: {val_f1:.4f}")

        # Free up GPU memory for more efficient use
        del model
        torch.cuda.empty_cache()
```

```
    # Average of performance metrics
    avg_accuracy = sum(fold_accuracies) / folds
    avg_f1 = sum(fold_f1_scores) / folds
    print(f"\nAverage Accuracy: {avg_accuracy:.4f}, Average F1: {avg_f1:.4f}")

    return avg_accuracy, avg_f1, fold_accuracies, fold_f1_scores, best_model_state
```

FinBERT

The first model experimented with is a 'base' one without any class weights adjustment and regularizations.

```
# Device
device = 'cuda' if torch.cuda.is_available() else 'cpu'
# Tokenizer
tokenizer = AutoTokenizer.from_pretrained("ProsusAI/finbert") # HuggingFace FinBERT
# Model (set as a function to reset weight)
def model_fn():
    return AutoModelForSequenceClassification.from_pretrained(
        "ProsusAI/finbert", num_labels=4, ignore_mismatched_sizes=True)
# Loss
loss_fn = nn.CrossEntropyLoss()

def optimizer_fn(model):
    return torch.optim.AdamW(model.parameters(), lr=2e-5)

avg_acc, avg_f1, fold_accs, fold_f1s, best_model_state = kfold_train(
    texts=texts,
    labels=labels,
    model_fn=model_fn,
    optimizer_fn=optimizer_fn,
    loss_fn=loss_fn,
    tokenizer=tokenizer,
    folds=5,
    epochs=5,
    batch_size=8,
    device=device
)
```

```
tokenizer_config.json: 100%  252/252 [00:00<00:00, 25.9kB/s]
config.json: 100%  758/758 [00:00<00:00, 18.1kB/s]
vocab.txt: 100%  232k/232k [00:00<00:00, 1.05MB/s]
special_tokens_map.json: 100%  112/112 [00:00<00:00, 10.6kB/s]

Fold 1/5
Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance, install the package with: `pip install hug
WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance, inst
pytorch_model.bin: 100%  438M/438M [00:01<00:00, 379MB/s]
Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance, install the package with: `pip install hug
WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance, inst
model.safetensors: 100%  438M/438M [00:01<00:00, 348MB/s]
  Epoch 1/5 | Train Loss: 1.2326 | Val Loss: 1.0307 | Val F1: 0.3511
  Epoch 2/5 | Train Loss: 0.8913 | Val Loss: 0.8211 | Val F1: 0.6369
  Epoch 3/5 | Train Loss: 0.6631 | Val Loss: 0.7479 | Val F1: 0.6744
  Epoch 4/5 | Train Loss: 0.4512 | Val Loss: 0.7333 | Val F1: 0.7226
  Epoch 5/5 | Train Loss: 0.2311 | Val Loss: 0.9514 | Val F1: 0.7257
  Fold Accuracy: 0.7233, F1: 0.7257

Fold 2/5
  Epoch 1/5 | Train Loss: 1.2173 | Val Loss: 1.0949 | Val F1: 0.3826
  Epoch 2/5 | Train Loss: 0.8634 | Val Loss: 0.7544 | Val F1: 0.7326
  Epoch 3/5 | Train Loss: 0.6444 | Val Loss: 0.7233 | Val F1: 0.7288
  Epoch 4/5 | Train Loss: 0.4258 | Val Loss: 0.8527 | Val F1: 0.6826
  Epoch 5/5 | Train Loss: 0.2810 | Val Loss: 0.7020 | Val F1: 0.7615
  Fold Accuracy: 0.7476, F1: 0.7615

Fold 3/5
  Epoch 1/5 | Train Loss: 1.2393 | Val Loss: 1.0745 | Val F1: 0.4319
  Epoch 2/5 | Train Loss: 0.8739 | Val Loss: 1.0976 | Val F1: 0.5886
  Epoch 3/5 | Train Loss: 0.6336 | Val Loss: 0.6828 | Val F1: 0.7397
  Epoch 4/5 | Train Loss: 0.3721 | Val Loss: 0.7443 | Val F1: 0.7548
  Epoch 5/5 | Train Loss: 0.2059 | Val Loss: 0.7383 | Val F1: 0.7427
  Fold Accuracy: 0.7317, F1: 0.7427

Fold 4/5
  Epoch 1/5 | Train Loss: 1.2391 | Val Loss: 1.0769 | Val F1: 0.3429
  Epoch 2/5 | Train Loss: 1.0059 | Val Loss: 0.8394 | Val F1: 0.5132
  Epoch 3/5 | Train Loss: 0.7465 | Val Loss: 0.8431 | Val F1: 0.6298
  Epoch 4/5 | Train Loss: 0.5507 | Val Loss: 0.7416 | Val F1: 0.7116
  Epoch 5/5 | Train Loss: 0.3946 | Val Loss: 0.9044 | Val F1: 0.6335
  Fold Accuracy: 0.6537, F1: 0.6335

Fold 5/5
  Epoch 1/5 | Train Loss: 1.2560 | Val Loss: 1.0136 | Val F1: 0.3177
  Epoch 2/5 | Train Loss: 0.9301 | Val Loss: 0.7210 | Val F1: 0.6614
  Epoch 3/5 | Train Loss: 0.6117 | Val Loss: 0.6398 | Val F1: 0.7534
  Epoch 4/5 | Train Loss: 0.3851 | Val Loss: 0.8422 | Val F1: 0.6886
  Epoch 5/5 | Train Loss: 0.2361 | Val Loss: 0.7672 | Val F1: 0.7054
  Fold Accuracy: 0.6878, F1: 0.7054

Average Accuracy: 0.7088, Average F1: 0.7414
```

```
save_path = "/content/drive/MyDrive/NLP CW/finbert_kfoldmodel1.pt"

torch.save({
    'model_state_dict': best_model_state,
    'avg_accuracy': avg_acc,
    'avg_f1': avg_f1
}, save_path)

print(f"Model saved: {save_path}")
```

```
Model saved: /content/drive/MyDrive/NLP CW/finbert_kfoldmodel1.pt
```

Overall performance is very good and similar to best SVM model. We obeserve steady improvement in Training and Validation Loss. However, there are signs of overfitting; in several folds we can observe a sudden increase in validation loss while training loss keeps decreasing. We observe variation in F1-score but in most fold the last epoch reach around the same amount. This suggests that the model might be sensitive to certain datapoints.

Since we observed validation loss increasing while training loss decreased, we consider early stopping. To mitigate overfitting as well we will consider dropout and weight decay.

Early Stopping

```
# kfold training function (including early stopping)
def kfold_train(texts, labels, model_fn, optimizer_fn, loss_fn, tokenizer, folds=5, epochs=3, batch_size=8, device=None):
    if device is None:
        device = 'cuda' if torch.cuda.is_available() else 'cpu'

    kf = KFold(n_splits=folds, shuffle=True, random_state=42) # Split data for each fold
    fold_accuracies = []
    fold_f1_scores = []

    # Variables to track best model across all folds
    best_overall_f1 = 0
    best_model_state = None

    for fold, (train_idx, val_idx) in enumerate(kf.split(texts)):
        print(f"\nFold {fold+1}/{folds}")

        # Prepare fold data
        train_texts = [texts[i] for i in train_idx]
        train_labels = [labels[i] for i in train_idx]
        val_texts = [texts[i] for i in val_idx]
        val_true_labels = [labels[i] for i in val_idx]

        # Create datasets and dataloaders
        train_dataset = FinancialRiskDataset(train_texts, train_labels, tokenizer)
        val_dataset = FinancialRiskDataset(val_texts, val_true_labels, tokenizer)
        train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
```

```
        val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)

        # Instantiate model
        model = model_fn().to(device)
        # Optimizer initialisation
        optimizer = optimizer_fn(model)

        best_fold_f1 = 0
        best_fold_model = None
        no_improvement_epochs = 0
        patience = 2

        # Training loop
        for epoch in range(epochs):
            train_loss = train(model, train_loader, optimizer, loss_fn, device)
            val_loss, val_f1, val_preds, _ = evaluate(model, val_loader, loss_fn, device)
            print(f"  Epoch {epoch+1}/{epochs} | Train Loss: {train_loss:.4f} | Val Loss: {val_loss:.4f} | Val F1: {val_f1:.4f}")

            # Tracking the best model per fold
            if val_f1 > best_fold_f1:
                best_fold_f1 = val_f1
                best_fold_model = model.state_dict().copy()

                # Best model across all folds
                if val_f1 > best_overall_f1:
                    best_overall_f1 = val_f1
                    best_model_state = model.state_dict().copy()
            else:
               no_improvement_epochs += 1

            # Early stopping
            if no_improvement_epochs >= patience:
                print(f"Early stopping at epoch {epoch+1}")
                break

        # Load the best model for this fold
        model.load_state_dict(best_fold_model)

        # Evaluation using function above (using f1 score because of imbalanced dataset)
        val_loss, val_f1, val_preds, _ = evaluate(model, val_loader, loss_fn, device)
        val_accuracy = accuracy_score(val_true_labels, val_preds)
        # Tracking fold results
        fold_accuracies.append(val_accuracy)
        fold_f1_scores.append(best_fold_f1) # save the model with best f1 rather than final epoch
        print(f"  Fold Accuracy: {val_accuracy:.4f}, F1: {val_f1:.4f}")

        # Free up GPU memory for more efficient use
        del model
        torch.cuda.empty_cache()
    # Average of performance metrics
    avg_accuracy = sum(fold_accuracies) / folds
    avg_f1 = sum(fold_f1_scores) / folds
    print(f"\nAverage Accuracy: {avg_accuracy:.4f}, Average F1: {avg_f1:.4f}")
```

```
    return avg_accuracy, avg_f1, fold_accuracies, fold_f1_scores, best_model_state


# Device
device = 'cuda' if torch.cuda.is_available() else 'cpu'
# Tokenizer
tokenizer = AutoTokenizer.from_pretrained("ProsusAI/finbert") # HuggingFace FinBERT
# Model (set as a function to reset weight)
def model_fn():
    return AutoModelForSequenceClassification.from_pretrained(
        "ProsusAI/finbert", num_labels=4, ignore_mismatched_sizes=True)
# Loss
loss_fn = nn.CrossEntropyLoss()

def optimizer_fn(model):
    return torch.optim.AdamW(model.parameters(), lr=2e-5)

avg_acc, avg_f1, fold_accs, fold_f1s, best_model_state = kfold_train(
    texts=texts,
    labels=labels,
    model_fn=model_fn,
    optimizer_fn=optimizer_fn,
    loss_fn=loss_fn,
    tokenizer=tokenizer,
    folds=5,
    epochs=5,
    batch_size=8,
    device=device
)
```

```
Fold 1/5
  Epoch 1/5 | Train Loss: 1.2027 | Val Loss: 1.0262 | Val F1: 0.5360
  Epoch 2/5 | Train Loss: 0.8496 | Val Loss: 0.7938 | Val F1: 0.6661
  Epoch 3/5 | Train Loss: 0.5788 | Val Loss: 0.7414 | Val F1: 0.7127
  Epoch 4/5 | Train Loss: 0.3824 | Val Loss: 0.7095 | Val F1: 0.7408
  Epoch 5/5 | Train Loss: 0.2521 | Val Loss: 0.8592 | Val F1: 0.7011
  Fold Accuracy: 0.6796, F1: 0.7011

Fold 2/5
  Epoch 1/5 | Train Loss: 1.1851 | Val Loss: 1.0824 | Val F1: 0.3795
  Epoch 2/5 | Train Loss: 0.9400 | Val Loss: 0.9627 | Val F1: 0.6210
  Epoch 3/5 | Train Loss: 0.7236 | Val Loss: 0.8314 | Val F1: 0.6644
  Epoch 4/5 | Train Loss: 0.5447 | Val Loss: 0.7666 | Val F1: 0.6772
  Epoch 5/5 | Train Loss: 0.3353 | Val Loss: 0.8065 | Val F1: 0.7216
  Fold Accuracy: 0.7087, F1: 0.7216

Fold 3/5
  Epoch 1/5 | Train Loss: 1.2528 | Val Loss: 1.2148 | Val F1: 0.2723
  Epoch 2/5 | Train Loss: 0.9036 | Val Loss: 0.8932 | Val F1: 0.5895
  Epoch 3/5 | Train Loss: 0.6367 | Val Loss: 0.8760 | Val F1: 0.6495
  Epoch 4/5 | Train Loss: 0.4412 | Val Loss: 0.9134 | Val F1: 0.6670
  Epoch 5/5 | Train Loss: 0.2872 | Val Loss: 1.1114 | Val F1: 0.6234
  Fold Accuracy: 0.6341, F1: 0.6234
```

```
Fold 4/5
  Epoch 1/5 | Train Loss: 1.2075 | Val Loss: 1.0144 | Val F1: 0.3732
  Epoch 2/5 | Train Loss: 0.9104 | Val Loss: 0.8714 | Val F1: 0.4576
  Epoch 3/5 | Train Loss: 0.6823 | Val Loss: 0.7179 | Val F1: 0.6962
  Epoch 4/5 | Train Loss: 0.4706 | Val Loss: 0.8916 | Val F1: 0.6633
  Epoch 5/5 | Train Loss: 0.3014 | Val Loss: 0.8581 | Val F1: 0.6744
Early stopping at epoch 5
  Fold Accuracy: 0.6634, F1: 0.6744

Fold 5/5
  Epoch 1/5 | Train Loss: 1.2559 | Val Loss: 1.0313 | Val F1: 0.2671
  Epoch 2/5 | Train Loss: 0.9488 | Val Loss: 0.9643 | Val F1: 0.4299
  Epoch 3/5 | Train Loss: 0.7051 | Val Loss: 0.6457 | Val F1: 0.6909
  Epoch 4/5 | Train Loss: 0.4958 | Val Loss: 0.6662 | Val F1: 0.7493
  Epoch 5/5 | Train Loss: 0.3782 | Val Loss: 0.5629 | Val F1: 0.7792
  Fold Accuracy: 0.7610, F1: 0.7792

Average Accuracy: 0.6894, Average F1: 0.7209
```

Early Stopping has not been triggered which suggest that the model is still improving. Instead of decreasing patience to 1 (which would be more aggressive), we would increase number of epochs to 7, this may leas to the model to perform better

We will add weight decay as well for better generalisability along with dropout

FinBERT (Early Stopping + Weight Decay)

More epochs (More opportunities for early stopping to be triggered)

```
# Device
device = 'cuda' if torch.cuda.is_available() else 'cpu'
# Tokenizer
tokenizer = AutoTokenizer.from_pretrained("ProsusAI/finbert") # HuggingFace FinBERT
# Model (set as a function to reset weight)
def model_fn():
    return AutoModelForSequenceClassification.from_pretrained(
        "ProsusAI/finbert", num_labels=4, ignore_mismatched_sizes=True)
# Loss
loss_fn = nn.CrossEntropyLoss()
# Added weight decay of 0.1
def optimizer_fn(model):
    return torch.optim.AdamW(model.parameters(), lr=2e-5, weight_decay=0.1)

avg_acc, avg_f1, fold_accs, fold_f1s, best_model_state = kfold_train(
    texts=texts,
    labels=labels,
    model_fn=model_fn,
    optimizer_fn=optimizer_fn,
    loss_fn=loss_fn,
    tokenizer=tokenizer,
    folds=5,
    epochs=7,
    batch_size=8,
```

```
    device=device
)
```

```
Fold 1/5
  Epoch 1/7 | Train Loss: 1.2959 | Val Loss: 1.1595 | Val F1: 0.2980
  Epoch 2/7 | Train Loss: 1.0388 | Val Loss: 0.9737 | Val F1: 0.4361
  Epoch 3/7 | Train Loss: 0.8847 | Val Loss: 0.9250 | Val F1: 0.4132
  Epoch 4/7 | Train Loss: 0.7126 | Val Loss: 0.8722 | Val F1: 0.5102
  Epoch 5/7 | Train Loss: 0.5176 | Val Loss: 0.9862 | Val F1: 0.5863
  Epoch 6/7 | Train Loss: 0.3210 | Val Loss: 0.9605 | Val F1: 0.6889
  Epoch 7/7 | Train Loss: 0.1225 | Val Loss: 1.0775 | Val F1: 0.7040
  Fold Accuracy: 0.6845, F1: 0.7040

Fold 2/5
  Epoch 1/7 | Train Loss: 1.1909 | Val Loss: 1.0595 | Val F1: 0.3997
  Epoch 2/7 | Train Loss: 0.8331 | Val Loss: 1.0056 | Val F1: 0.4694
  Epoch 3/7 | Train Loss: 0.5924 | Val Loss: 0.7259 | Val F1: 0.6605
  Epoch 4/7 | Train Loss: 0.3592 | Val Loss: 0.8378 | Val F1: 0.7259
  Epoch 5/7 | Train Loss: 0.2260 | Val Loss: 1.0863 | Val F1: 0.6560
  Epoch 6/7 | Train Loss: 0.1271 | Val Loss: 0.9907 | Val F1: 0.7238
Early stopping at epoch 6
  Fold Accuracy: 0.7039, F1: 0.7238

Fold 3/5
  Epoch 1/7 | Train Loss: 1.1883 | Val Loss: 1.0847 | Val F1: 0.4854
  Epoch 2/7 | Train Loss: 0.8828 | Val Loss: 0.9445 | Val F1: 0.6078
  Epoch 3/7 | Train Loss: 0.6323 | Val Loss: 0.8932 | Val F1: 0.6826
  Epoch 4/7 | Train Loss: 0.4453 | Val Loss: 0.8540 | Val F1: 0.6973
  Epoch 5/7 | Train Loss: 0.2595 | Val Loss: 0.8015 | Val F1: 0.7440
  Epoch 6/7 | Train Loss: 0.1709 | Val Loss: 1.0444 | Val F1: 0.6995
  Epoch 7/7 | Train Loss: 0.0823 | Val Loss: 0.9682 | Val F1: 0.7254
Early stopping at epoch 7
  Fold Accuracy: 0.7171, F1: 0.7254

Fold 4/5
  Epoch 1/7 | Train Loss: 1.2193 | Val Loss: 1.0548 | Val F1: 0.3190
  Epoch 2/7 | Train Loss: 0.9617 | Val Loss: 0.8564 | Val F1: 0.5005
  Epoch 3/7 | Train Loss: 0.7200 | Val Loss: 0.7796 | Val F1: 0.6486
  Epoch 4/7 | Train Loss: 0.5488 | Val Loss: 0.7181 | Val F1: 0.7297
  Epoch 5/7 | Train Loss: 0.3427 | Val Loss: 0.7452 | Val F1: 0.7125
  Epoch 6/7 | Train Loss: 0.1894 | Val Loss: 0.7587 | Val F1: 0.7736
  Epoch 7/7 | Train Loss: 0.0950 | Val Loss: 0.9260 | Val F1: 0.7202
Early stopping at epoch 7
  Fold Accuracy: 0.6878, F1: 0.7202

Fold 5/5
  Epoch 1/7 | Train Loss: 1.2995 | Val Loss: 1.1379 | Val F1: 0.2489
  Epoch 2/7 | Train Loss: 1.0492 | Val Loss: 0.7944 | Val F1: 0.6737
  Epoch 3/7 | Train Loss: 0.7346 | Val Loss: 0.7151 | Val F1: 0.6669
  Epoch 4/7 | Train Loss: 0.5136 | Val Loss: 0.5534 | Val F1: 0.7923
  Epoch 5/7 | Train Loss: 0.3484 | Val Loss: 0.6269 | Val F1: 0.7454
Early stopping at epoch 5
  Fold Accuracy: 0.7220, F1: 0.7454

Average Accuracy: 0.7030, Average F1: 0.7480
```

```
save_path = "/content/drive/MyDrive/NLP CW/finbert_kfoldmodel2.pt"

torch.save({
    'model_state_dict': best_model_state,
    'avg_accuracy': avg_acc,
    'avg_f1': avg_f1
}, save_path)

print(f"Model saved: {save_path}")
```

```
Model saved: /content/drive/MyDrive/NLP CW/finbert_kfoldmodel2.pt
```

FinBERT (Early Stopping + Weight Decay + Dropout)

We experiment with dropout to see if there are any improvement in performance

```
# Device
device = 'cuda' if torch.cuda.is_available() else 'cpu'
# Tokenizer
tokenizer = AutoTokenizer.from_pretrained("ProsusAI/finbert") # HuggingFace FinBERT
# Model (set as a function to reset weight) + Added Dropout
def model_fn(dropout_rate=0.3):
    model = AutoModelForSequenceClassification.from_pretrained(
        "ProsusAI/finbert",
        num_labels=4,
        ignore_mismatched_sizes=True
    )
    # Modifying the dropout rate of the classifier
    model.classifier.dropout = nn.Dropout(p=dropout_rate)
    return model
# Loss
loss_fn = nn.CrossEntropyLoss()
# Added weight decay of 0.1
def optimizer_fn(model):
    return torch.optim.AdamW(model.parameters(), lr=2e-5, weight_decay=0.1)

avg_acc, avg_f1, fold_accs, fold_f1s, best_model_state = kfold_train(
    texts=texts,
    labels=labels,
    model_fn=lambda: model_fn(dropout_rate=0.3),
    optimizer_fn=optimizer_fn,
    loss_fn=loss_fn,
    tokenizer=tokenizer,
    folds=5,
    epochs=7,
    batch_size=8,
    device=device
)
```

```
 Fold 1/5
   Epoch 1/7 | Train Loss: 1.1817 | Val Loss: 0.9459 | Val F1: 0.3842
```

```
  Epoch 2/7 | Train Loss: 0.8668 | Val Loss: 0.7851 | Val F1: 0.6178
  Epoch 3/7 | Train Loss: 0.6429 | Val Loss: 0.7684 | Val F1: 0.6927
  Epoch 4/7 | Train Loss: 0.4334 | Val Loss: 0.7769 | Val F1: 0.7251
  Epoch 5/7 | Train Loss: 0.2514 | Val Loss: 0.8062 | Val F1: 0.7069
  Epoch 6/7 | Train Loss: 0.1637 | Val Loss: 0.7709 | Val F1: 0.7675
  Epoch 7/7 | Train Loss: 0.0815 | Val Loss: 0.9955 | Val F1: 0.7217
Early stopping at epoch 7
  Fold Accuracy: 0.6990, F1: 0.7217

Fold 2/5
  Epoch 1/7 | Train Loss: 1.2048 | Val Loss: 1.0947 | Val F1: 0.3308
  Epoch 2/7 | Train Loss: 0.8780 | Val Loss: 0.8613 | Val F1: 0.5798
  Epoch 3/7 | Train Loss: 0.6427 | Val Loss: 0.7488 | Val F1: 0.7270
  Epoch 4/7 | Train Loss: 0.3812 | Val Loss: 0.9178 | Val F1: 0.6282
  Epoch 5/7 | Train Loss: 0.2770 | Val Loss: 0.8140 | Val F1: 0.7371
  Epoch 6/7 | Train Loss: 0.1564 | Val Loss: 0.9797 | Val F1: 0.6961
Early stopping at epoch 6
  Fold Accuracy: 0.6942, F1: 0.6961

Fold 3/5
  Epoch 1/7 | Train Loss: 1.2165 | Val Loss: 1.2190 | Val F1: 0.3394
  Epoch 2/7 | Train Loss: 0.9475 | Val Loss: 0.8042 | Val F1: 0.6657
  Epoch 3/7 | Train Loss: 0.6433 | Val Loss: 0.8205 | Val F1: 0.6488
  Epoch 4/7 | Train Loss: 0.4682 | Val Loss: 0.7137 | Val F1: 0.7245
  Epoch 5/7 | Train Loss: 0.3231 | Val Loss: 0.7035 | Val F1: 0.7312
  Epoch 6/7 | Train Loss: 0.1914 | Val Loss: 0.8253 | Val F1: 0.7448
  Epoch 7/7 | Train Loss: 0.1618 | Val Loss: 0.8396 | Val F1: 0.7183
Early stopping at epoch 7
  Fold Accuracy: 0.7171, F1: 0.7183

Fold 4/5
  Epoch 1/7 | Train Loss: 1.1983 | Val Loss: 1.0256 | Val F1: 0.4388
  Epoch 2/7 | Train Loss: 0.8965 | Val Loss: 0.7833 | Val F1: 0.6840
  Epoch 3/7 | Train Loss: 0.6239 | Val Loss: 0.8329 | Val F1: 0.6253
  Epoch 4/7 | Train Loss: 0.4006 | Val Loss: 0.7166 | Val F1: 0.6954
  Epoch 5/7 | Train Loss: 0.2539 | Val Loss: 0.6512 | Val F1: 0.7823
  Epoch 6/7 | Train Loss: 0.1169 | Val Loss: 1.0551 | Val F1: 0.7143
Early stopping at epoch 6
  Fold Accuracy: 0.6683, F1: 0.7143

Fold 5/5
  Epoch 1/7 | Train Loss: 1.2182 | Val Loss: 1.0698 | Val F1: 0.4372
  Epoch 2/7 | Train Loss: 0.8917 | Val Loss: 0.7968 | Val F1: 0.6380
  Epoch 3/7 | Train Loss: 0.6079 | Val Loss: 0.6182 | Val F1: 0.7130
  Epoch 4/7 | Train Loss: 0.3776 | Val Loss: 0.5429 | Val F1: 0.7977
  Epoch 5/7 | Train Loss: 0.2256 | Val Loss: 0.6747 | Val F1: 0.7931
  Epoch 6/7 | Train Loss: 0.1414 | Val Loss: 0.6304 | Val F1: 0.8005
  Epoch 7/7 | Train Loss: 0.0895 | Val Loss: 0.7224 | Val F1: 0.8137
  Fold Accuracy: 0.7854, F1: 0.8137

Average Accuracy: 0.7128, Average F1: 0.7691
```

```
save_path = "/content/drive/MyDrive/NLP CW/finbert_kfoldmodel3.pt"

torch.save({
    'model_state_dict': best_model_state,
    'avg_accuracy': avg_acc,
```

```
    'avg_f1': avg_f1
}, save_path)

print(f"Model saved: {save_path}")
```

```
Model saved: /content/drive/MyDrive/NLP CW/finbert_kfoldmodel3.pt
```

Having applied multiple techniques to address the limitations of the dataset; we have obtained relatively good performance from the last model of FinBERT. In the last fold, we achived accuracy 0.78 and f1 of 081, which is a great improvement. To avoid excessive tuning and experience diminishing return, we stop with FinBERT here to address other aspects of the project. One more thing that could be experimented with is adding adjusted class weights to the loss function (if we have time, we will explore this)

FinBERT (Regularizations + Class Weights)

```
# Device
device = 'cuda' if torch.cuda.is_available() else 'cpu'
# Tokenizer
tokenizer = AutoTokenizer.from_pretrained("ProsusAI/finbert") # HuggingFace FinBERT
# Model (set as a function to reset weight) + Added Dropout
def model_fn(dropout_rate=0.3):
    model = AutoModelForSequenceClassification.from_pretrained(
        "ProsusAI/finbert",
        num_labels=4,
        ignore_mismatched_sizes=True
    )
    # Modifying the dropout rate of the classifier
    model.classifier.dropout = nn.Dropout(p=dropout_rate)
    return model

# Loss (adjusted based on class weights)
classes = np.unique(labels)
class_weights = compute_class_weight(class_weight='balanced', classes=classes, y=labels)
class_weights_tensor = torch.tensor(class_weights, dtype=torch.float).to(device)
loss_fn = nn.CrossEntropyLoss(weight=class_weights_tensor)

# Added weight decay of 0.1
def optimizer_fn(model):
    return torch.optim.AdamW(model.parameters(), lr=2e-5, weight_decay=0.1)

avg_acc, avg_f1, fold_accs, fold_f1s, best_model_state = kfold_train(
    texts=texts,
    labels=labels,
    model_fn=lambda: model_fn(dropout_rate=0.3),
    optimizer_fn=optimizer_fn,
    loss_fn=loss_fn,
    tokenizer=tokenizer,
    folds=5,
    epochs=7,
    batch_size=8,
```

```
    device=device
)
```

```
Fold 1/5
  Epoch 1/7 | Train Loss: 1.2615 | Val Loss: 1.1281 | Val F1: 0.4388
  Epoch 2/7 | Train Loss: 0.9292 | Val Loss: 0.8198 | Val F1: 0.6737
  Epoch 3/7 | Train Loss: 0.6564 | Val Loss: 0.5439 | Val F1: 0.7398
  Epoch 4/7 | Train Loss: 0.4137 | Val Loss: 0.5332 | Val F1: 0.7816
  Epoch 5/7 | Train Loss: 0.2812 | Val Loss: 0.8059 | Val F1: 0.7263
  Epoch 6/7 | Train Loss: 0.1947 | Val Loss: 0.7316 | Val F1: 0.7504
Early stopping at epoch 6
  Fold Accuracy: 0.7379, F1: 0.7504

Fold 2/5
  Epoch 1/7 | Train Loss: 1.2596 | Val Loss: 1.0682 | Val F1: 0.4787
  Epoch 2/7 | Train Loss: 0.9413 | Val Loss: 0.7805 | Val F1: 0.6441
  Epoch 3/7 | Train Loss: 0.6071 | Val Loss: 0.6957 | Val F1: 0.7318
  Epoch 4/7 | Train Loss: 0.4097 | Val Loss: 0.6171 | Val F1: 0.7299
  Epoch 5/7 | Train Loss: 0.3174 | Val Loss: 0.7628 | Val F1: 0.7019
Early stopping at epoch 5
  Fold Accuracy: 0.6602, F1: 0.7019

Fold 3/5
  Epoch 1/7 | Train Loss: 1.2813 | Val Loss: 1.1724 | Val F1: 0.3727
  Epoch 2/7 | Train Loss: 0.9437 | Val Loss: 0.8414 | Val F1: 0.5472
  Epoch 3/7 | Train Loss: 0.6346 | Val Loss: 0.6908 | Val F1: 0.6644
  Epoch 4/7 | Train Loss: 0.4151 | Val Loss: 0.6067 | Val F1: 0.6996
  Epoch 5/7 | Train Loss: 0.2498 | Val Loss: 0.6667 | Val F1: 0.6789
  Epoch 6/7 | Train Loss: 0.1435 | Val Loss: 0.7566 | Val F1: 0.7407
  Epoch 7/7 | Train Loss: 0.1321 | Val Loss: 0.7618 | Val F1: 0.7264
Early stopping at epoch 7
  Fold Accuracy: 0.7171, F1: 0.7264

Fold 4/5
  Epoch 1/7 | Train Loss: 1.2908 | Val Loss: 1.2701 | Val F1: 0.2406
  Epoch 2/7 | Train Loss: 1.0304 | Val Loss: 0.8812 | Val F1: 0.5874
  Epoch 3/7 | Train Loss: 0.7569 | Val Loss: 0.7147 | Val F1: 0.6300
  Epoch 4/7 | Train Loss: 0.5012 | Val Loss: 0.5668 | Val F1: 0.7032
  Epoch 5/7 | Train Loss: 0.3331 | Val Loss: 0.6125 | Val F1: 0.7172
  Epoch 6/7 | Train Loss: 0.2593 | Val Loss: 0.5851 | Val F1: 0.7534
  Epoch 7/7 | Train Loss: 0.1736 | Val Loss: 0.6288 | Val F1: 0.7890
  Fold Accuracy: 0.7610, F1: 0.7890

Fold 5/5
  Epoch 1/7 | Train Loss: 1.2393 | Val Loss: 1.2215 | Val F1: 0.3967
  Epoch 2/7 | Train Loss: 0.8548 | Val Loss: 0.8399 | Val F1: 0.5923
  Epoch 3/7 | Train Loss: 0.5643 | Val Loss: 0.7745 | Val F1: 0.6654
  Epoch 4/7 | Train Loss: 0.3976 | Val Loss: 0.8455 | Val F1: 0.7262
  Epoch 5/7 | Train Loss: 0.2433 | Val Loss: 0.7577 | Val F1: 0.7451
  Epoch 6/7 | Train Loss: 0.1632 | Val Loss: 0.8848 | Val F1: 0.7486
  Epoch 7/7 | Train Loss: 0.1191 | Val Loss: 0.8655 | Val F1: 0.7722
  Fold Accuracy: 0.7512, F1: 0.7722

Average Accuracy: 0.7255, Average F1: 0.7630
```

```
save_path = "/content/drive/MyDrive/NLP CW/finbert_kfoldmodel4.pt"

torch.save({
    'model_state_dict': best_model_state,
    'avg_accuracy': avg_acc,
    'avg_f1': avg_f1
}, save_path)

print(f"Model saved: {save_path}")
```

```
Model saved: /content/drive/MyDrive/NLP CW/finbert_kfoldmodel4.pt
```

RoBERTa

To implement RoBERTa (a transformer model for NLP tasks without any domain-specific pretraining), we will reuse the same functions from FinBERT but with different model function and tokenizer. We will use the augmented data and implement the model without any regularization first and finally with the regularizations that led to the best FinBERT model (i.e Early Stopping, Weight Decay and Dropout). We might fine tune the model to our data in a third model.

```
# Device
device = 'cuda' if torch.cuda.is_available() else 'cpu'
# Tokenizer (auto tokenizer set as 'roberta-base' to select the right one)
tokenizer = AutoTokenizer.from_pretrained("roberta-base")
# Model (set as a function to reset weight)
def model_fn():
    return AutoModelForSequenceClassification.from_pretrained(
        "roberta-base", num_labels=4, ignore_mismatched_sizes=True)
# Loss
loss_fn = nn.CrossEntropyLoss()

def optimizer_fn(model):
    return torch.optim.AdamW(model.parameters(), lr=2e-5)

avg_acc, avg_f1, fold_accs, fold_f1s, best_model_state = kfold_train(
    texts=texts,
    labels=labels,
    model_fn=model_fn,
    optimizer_fn=optimizer_fn,
    loss_fn=loss_fn,
    tokenizer=tokenizer,
    folds=5,
    epochs=5,
    batch_size=8,
    device=device
)
```

```
 Fold 1/5
   Epoch 1/5 | Train Loss: 1.1783 | Val Loss: 0.9800 | Val F1: 0.2881
   Epoch 2/5 | Train Loss: 0.9121 | Val Loss: 0.8196 | Val F1: 0.5789
```

```
  Epoch 3/5 | Train Loss: 0.6807 | Val Loss: 0.7409 | Val F1: 0.6528
  Epoch 4/5 | Train Loss: 0.4938 | Val Loss: 0.7222 | Val F1: 0.6642
  Epoch 5/5 | Train Loss: 0.3217 | Val Loss: 0.8373 | Val F1: 0.6459
  Fold Accuracy: 0.6553, F1: 0.6459

Fold 2/5
  Epoch 1/5 | Train Loss: 1.1669 | Val Loss: 0.8815 | Val F1: 0.4484
  Epoch 2/5 | Train Loss: 0.8683 | Val Loss: 0.8004 | Val F1: 0.5618
  Epoch 3/5 | Train Loss: 0.6233 | Val Loss: 0.8187 | Val F1: 0.6419
  Epoch 4/5 | Train Loss: 0.3978 | Val Loss: 0.7175 | Val F1: 0.7324
  Epoch 5/5 | Train Loss: 0.2685 | Val Loss: 0.8772 | Val F1: 0.7082
  Fold Accuracy: 0.6796, F1: 0.7082

Fold 3/5
  Epoch 1/5 | Train Loss: 1.1455 | Val Loss: 1.0253 | Val F1: 0.3645
  Epoch 2/5 | Train Loss: 0.8749 | Val Loss: 0.8906 | Val F1: 0.5973
  Epoch 3/5 | Train Loss: 0.6668 | Val Loss: 0.8014 | Val F1: 0.6050
  Epoch 4/5 | Train Loss: 0.4941 | Val Loss: 0.7797 | Val F1: 0.6665
  Epoch 5/5 | Train Loss: 0.3048 | Val Loss: 0.8497 | Val F1: 0.6961
  Fold Accuracy: 0.6976, F1: 0.6961

Fold 4/5
  Epoch 1/5 | Train Loss: 1.1543 | Val Loss: 0.9960 | Val F1: 0.3754
  Epoch 2/5 | Train Loss: 0.9086 | Val Loss: 0.8620 | Val F1: 0.5382
  Epoch 3/5 | Train Loss: 0.6838 | Val Loss: 0.6750 | Val F1: 0.6834
  Epoch 4/5 | Train Loss: 0.5484 | Val Loss: 0.7800 | Val F1: 0.6835
  Epoch 5/5 | Train Loss: 0.3740 | Val Loss: 0.6631 | Val F1: 0.7530
  Fold Accuracy: 0.7366, F1: 0.7530

Fold 5/5
  Epoch 1/5 | Train Loss: 1.1688 | Val Loss: 1.0857 | Val F1: 0.3000
  Epoch 2/5 | Train Loss: 0.8311 | Val Loss: 0.7219 | Val F1: 0.7302
  Epoch 3/5 | Train Loss: 0.6132 | Val Loss: 0.7316 | Val F1: 0.6918
  Epoch 4/5 | Train Loss: 0.4131 | Val Loss: 0.5832 | Val F1: 0.7577
  Epoch 5/5 | Train Loss: 0.2797 | Val Loss: 0.9387 | Val F1: 0.6811
  Fold Accuracy: 0.6878, F1: 0.6811

Average Accuracy: 0.6914, Average F1: 0.7207
```

```
save_path = "/content/drive/MyDrive/NLP CW/roberta_kfoldmodel1.pt"

torch.save({
    'model_state_dict': best_model_state,
    'avg_accuracy': avg_acc,
    'avg_f1': avg_f1
}, save_path)

print(f"Model saved: {save_path}")
```

```
Model saved: /content/drive/MyDrive/NLP CW/roberta_kfoldmodel1.pt
```

RoBERTa (Early Stopping + Weight Decay + Dropout)

```
# Early Stopping training function
def kfold_train_es(texts, labels, model_fn, optimizer_fn, loss_fn, tokenizer, folds=5, epochs=3, batch_size=8, device=None):
    if device is None:
        device = 'cuda' if torch.cuda.is_available() else 'cpu'

    kf = KFold(n_splits=folds, shuffle=True, random_state=42) # Split data for each fold
    fold_accuracies = []
    fold_f1_scores = []

    # Variables to track best model across all folds
    best_overall_f1 = 0
    best_model_state = None

    for fold, (train_idx, val_idx) in enumerate(kf.split(texts)):
        print(f"\nFold {fold+1}/{folds}")

        # Prepare fold data
        train_texts = [texts[i] for i in train_idx]
        train_labels = [labels[i] for i in train_idx]
        val_texts = [texts[i] for i in val_idx]
        val_true_labels = [labels[i] for i in val_idx]

        # Create datasets and dataloaders
        train_dataset = FinancialRiskDataset(train_texts, train_labels, tokenizer)
        val_dataset = FinancialRiskDataset(val_texts, val_true_labels, tokenizer)
        train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
        val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)

        # Instantiate model
        model = model_fn().to(device)
        # Optimizer initialisation
        optimizer = optimizer_fn(model)

        best_fold_f1 = 0
        best_fold_model = None
        no_improvement_epochs = 0
        patience = 2

        # Training loop
        for epoch in range(epochs):
            train_loss = train(model, train_loader, optimizer, loss_fn, device)
            val_loss, val_f1, val_preds, _ = evaluate(model, val_loader, loss_fn, device)
            print(f"  Epoch {epoch+1}/{epochs} | Train Loss: {train_loss:.4f} | Val Loss: {val_loss:.4f} | Val F1: {val_f1:.4f}")

            # Tracking the best model per fold
            if val_f1 > best_fold_f1:
                best_fold_f1 = val_f1
                best_fold_model = model.state_dict().copy()

                # Best model across all folds
                if val_f1 > best_overall_f1:
                    best_overall_f1 = val_f1
                    best_model_state = model.state_dict().copy()
            else:
```

```
                no_improvement_epochs += 1

            # Early stopping
            if no_improvement_epochs >= patience:
                print(f"Early stopping at epoch {epoch+1}")
                break

        # Load the best model for this fold
        model.load_state_dict(best_fold_model)

        # Evaluation using function above (using f1 score because of imbalanced dataset)
        val_loss, val_f1, val_preds, _ = evaluate(model, val_loader, loss_fn, device)
        val_accuracy = accuracy_score(val_true_labels, val_preds)
        # Tracking fold results
        fold_accuracies.append(val_accuracy)
        fold_f1_scores.append(best_fold_f1) # save the model with best f1 rather than final epoch
        print(f"  Fold Accuracy: {val_accuracy:.4f}, F1: {val_f1:.4f}")

        # Free up GPU memory for more efficient use
        del model
        torch.cuda.empty_cache()
    # Average of performance metrics
    avg_accuracy = sum(fold_accuracies) / folds
    avg_f1 = sum(fold_f1_scores) / folds
    print(f"\nAverage Accuracy: {avg_accuracy:.4f}, Average F1: {avg_f1:.4f}")

    return avg_accuracy, avg_f1, fold_accuracies, fold_f1_scores, best_model_state


# Device
device = 'cuda' if torch.cuda.is_available() else 'cpu'
# Tokenizer
tokenizer = AutoTokenizer.from_pretrained("roberta-base")
# Model (set as a function to reset weight) + Added Dropout
def model_fn(dropout_rate=0.3):
    model = AutoModelForSequenceClassification.from_pretrained(
        "roberta-base",
        num_labels=4,
        ignore_mismatched_sizes=True
    )
    # Modifying the dropout rate of the classifier
    model.classifier.dropout = nn.Dropout(p=dropout_rate)
    return model
# Loss
loss_fn = nn.CrossEntropyLoss()
# Added weight decay of 0.1
def optimizer_fn(model):
    return torch.optim.AdamW(model.parameters(), lr=2e-5, weight_decay=0.1)

avg_acc, avg_f1, fold_accs, fold_f1s, best_model_state = kfold_train_es(
    texts=texts,
    labels=labels,
    model_fn=lambda: model_fn(dropout_rate=0.3),
    optimizer_fn=optimizer_fn,
```

```
    loss_fn=loss_fn,
    tokenizer=tokenizer,
    folds=5,
    epochs=7,
    batch_size=8,
    device=device
)
```

```
Fold 1/5
  Epoch 1/7 | Train Loss: 1.2305 | Val Loss: 0.9676 | Val F1: 0.2949
  Epoch 2/7 | Train Loss: 0.8942 | Val Loss: 0.9958 | Val F1: 0.5543
  Epoch 3/7 | Train Loss: 0.6834 | Val Loss: 0.7273 | Val F1: 0.6559
  Epoch 4/7 | Train Loss: 0.5586 | Val Loss: 0.6214 | Val F1: 0.7129
  Epoch 5/7 | Train Loss: 0.3556 | Val Loss: 0.7574 | Val F1: 0.7174
  Epoch 6/7 | Train Loss: 0.2862 | Val Loss: 0.7984 | Val F1: 0.6994
  Epoch 7/7 | Train Loss: 0.1583 | Val Loss: 1.1749 | Val F1: 0.6575
Early stopping at epoch 7
  Fold Accuracy: 0.6699, F1: 0.6575

Fold 2/5
  Epoch 1/7 | Train Loss: 1.1803 | Val Loss: 0.9598 | Val F1: 0.5018
  Epoch 2/7 | Train Loss: 0.9254 | Val Loss: 0.7392 | Val F1: 0.5009
  Epoch 3/7 | Train Loss: 0.7366 | Val Loss: 0.7127 | Val F1: 0.6953
  Epoch 4/7 | Train Loss: 0.5078 | Val Loss: 0.7204 | Val F1: 0.7298
  Epoch 5/7 | Train Loss: 0.3561 | Val Loss: 0.7136 | Val F1: 0.7608
  Epoch 6/7 | Train Loss: 0.2346 | Val Loss: 0.9500 | Val F1: 0.7638
  Epoch 7/7 | Train Loss: 0.2156 | Val Loss: 0.8796 | Val F1: 0.7357
Early stopping at epoch 7
  Fold Accuracy: 0.7184, F1: 0.7357

Fold 3/5
  Epoch 1/7 | Train Loss: 1.1848 | Val Loss: 1.0437 | Val F1: 0.3712
  Epoch 2/7 | Train Loss: 0.9832 | Val Loss: 0.9539 | Val F1: 0.3865
  Epoch 3/7 | Train Loss: 0.8579 | Val Loss: 0.8623 | Val F1: 0.5238
  Epoch 4/7 | Train Loss: 0.6995 | Val Loss: 0.7066 | Val F1: 0.6705
  Epoch 5/7 | Train Loss: 0.5930 | Val Loss: 0.7317 | Val F1: 0.7116
  Epoch 6/7 | Train Loss: 0.4048 | Val Loss: 0.9051 | Val F1: 0.6814
  Epoch 7/7 | Train Loss: 0.2693 | Val Loss: 0.9160 | Val F1: 0.7126
  Fold Accuracy: 0.7171, F1: 0.7126

Fold 4/5
  Epoch 1/7 | Train Loss: 1.1859 | Val Loss: 1.0707 | Val F1: 0.3849
  Epoch 2/7 | Train Loss: 0.8880 | Val Loss: 0.9013 | Val F1: 0.5519
  Epoch 3/7 | Train Loss: 0.6955 | Val Loss: 0.7102 | Val F1: 0.7151
  Epoch 4/7 | Train Loss: 0.5028 | Val Loss: 0.6370 | Val F1: 0.7530
  Epoch 5/7 | Train Loss: 0.3653 | Val Loss: 0.7334 | Val F1: 0.7521
  Epoch 6/7 | Train Loss: 0.1923 | Val Loss: 0.7684 | Val F1: 0.7497
Early stopping at epoch 6
  Fold Accuracy: 0.7220, F1: 0.7497

Fold 5/5
  Epoch 1/7 | Train Loss: 1.1967 | Val Loss: 1.0589 | Val F1: 0.3013
  Epoch 2/7 | Train Loss: 0.9820 | Val Loss: 1.0157 | Val F1: 0.3661
  Epoch 3/7 | Train Loss: 0.9023 | Val Loss: 0.9536 | Val F1: 0.5681
  Epoch 4/7 | Train Loss: 0.7190 | Val Loss: 0.9589 | Val F1: 0.6769
  Epoch 5/7 | Train Loss: 0.5576 | Val Loss: 0.7005 | Val F1: 0.7151
  Epoch 6/7 | Train Loss: 0.3863 | Val Loss: 0.6826 | Val F1: 0.7538
```

```
        Epoch 7/7 | Train Loss: 0.2764 | Val Loss: 0.8423 | Val F1: 0.7563
        Fold Accuracy: 0.7317, F1: 0.7563

     Average Accuracy: 0.7118, Average F1: 0.7406
```

```
save_path = "/content/drive/MyDrive/NLP CW/roberta_kfoldmodel2.pt"

torch.save({
    'model_state_dict': best_model_state,
    'avg_accuracy': avg_acc,
    'avg_f1': avg_f1
}, save_path)

print(f"Model saved: {save_path}")
```

```
Model saved: /content/drive/MyDrive/NLP CW/roberta_kfoldmodel2.pt
```

Key Phrase Extraction

After successfully building and evaluating our FinBERT model for risk severity classification, we focus on keyphrase extraction to identify the specific language patterns and terms associated with different risk severity levels. This addresses the second component of our project: extracting key risk factors from financial disclosures.

While our classification model excels at predicting severity, understanding *why* a disclosure receives a particular severity rating is also important for financial analysis. By extracting keyphrases, we can:

1. Identify specific terms and phrases that signal different risk severity levels
2. Compare these linguistically important terms with the features our model focuses on (via SHAP/LIME)
3. Develop a more interpretable risk assessment framework that combines prediction with explanation

We will use KeyBERT as our main model (but may explore others like YAKE). This would hopefully enhance explainability of our risk analysis pipeline and be helpful in decision making.

KeyBERT (+ FinBERT)

```
from keybert import KeyBERT


# We will be testing KeyBERT to monitor if it is performing actually well (i.e extracting useful keywords)
# For each severity levels, we will apply the model on a few samples
# We basically run the analysis by severity level to understand distinct keyphrases for each risk category


# We use the same pretrained FinBERT model to ensure extracted key phrases aligns with financial terminologies
keybert_model = KeyBERT(model='ProsusAI/finbert')
```

```
WARNING:sentence_transformers.SentenceTransformer:No sentence-transformers model found with name ProsusAI/finbert. Creating a new one with mean pooling.
config.json: 100% 758/758 [00:00<00:00, 19.5kB/s]
Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance, install the package with: `pip install hug
WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance, inst
pytorch_model.bin: 100% 438M/438M [00:02<00:00, 243MB/s]
Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance, install the package with: `pip install hug
WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance, inst
model.safetensors: 12% 52.4M/438M [00:00<00:04, 90.8MB/s]
tokenizer_config.json: 100% 252/252 [00:00<00:00, 14.3kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 5.59MB/s]
special_tokens_map.json: 100% 112/112 [00:00<00:00, 4.20kB/s]
```

```
# Extract keyphrases by severity level (Bi-Grams as we've seen in EDA Bi-grams seem to carry more semantic meaning)
for risk in ['HIGH', 'MEDIUM', 'LOW', 'NONE']:
    print(f"\n===== Keyphrases for {risk} Risk Severity =====")

    # Sample 5-10 documents randomly from each severity level
    risk_docs = df[df['risk_severity'] == risk]['input'].sample(5).values

    for i, doc in enumerate(risk_docs):
        print(f"\nDocument {i+1} excerpt: {doc[:100]}...")

        keyphrases = keybert_model.extract_keywords(
            doc,
            keyphrase_ngram_range=(1, 2),
            use_mmr=True,
            diversity=0.7,
            top_n=5
        )

        print("Keyphrases:")
        for phrase, score in keyphrases:
            print(f"- {phrase}: {score:.4f}")
```

```
- mitigating: 0.4357
```

```
- months company: 0.4346
- effect settlement: 0.4228
- decrease carrying: 0.2749

Document 3 excerpt: and a gain on sale of $4.3 million in the three months ended September 30, 2023, compared to a loss ...
Keyphrases:
- segments reported: 0.6499
- operating results: 0.5786
- performed reporting: 0.4820
- decline operating: 0.3406
- increased competition: 0.2766

Document 4 excerpt: ...lives over their estimated useful lives, which range from 3 to 15 years. We review our goodwill a...
Keyphrases:
- receivable balances: 0.7257
- materially expressed: 0.5222
- believe critical: 0.4904
- pandemic: 0.4787
- increased costs: 0.2952

Document 5 excerpt: Item 7.01 Regulation FD Disclosure.

On July 22, 2022, the Company issued a press release announcing...
Keyphrases:
- 2022 dividend: 0.7345
- innovating delivering: 0.5816
- pitney bowes: 0.5210
- materially expressed: 0.4425
- increased adoption: 0.3380

===== Keyphrases for LOW Risk Severity =====

Document 1 excerpt: Item 1.01 Entry into a Material Definitive Agreement.

On June 16, 2022, the Company entered into a ...
Keyphrases:
- include covenants: 0.7765
- indemnification rights: 0.6663
- june 16: 0.5502
- thereto: 0.4247
- failure company: 0.3459

Document 2 excerpt: The Company has no exposure to foreign currency exchange rate risk for its U.S. dollar denominated a
```

```
# Extract keyphrases by severity level (Uni-Grams)
for risk in ['HIGH', 'MEDIUM', 'LOW', 'NONE']:
    print(f"\n===== Keyphrases for {risk} Risk Severity =====")

    # Sample 5-10 documents randomly from each severity level
    risk_docs = df[df['risk_severity'] == risk]['input'].sample(5).values

    for i, doc in enumerate(risk_docs):
    # Display first 100 characters of the document for aesthetic purposes
        print(f"\nDocument {i+1} excerpt: {doc[:100]}...")

        keyphrases = keybert_model.extract_keywords(
            doc,
            keyphrase_ngram_range=(1, 1),
```

```
        use_mmr=True,
        diversity=0.7,
        top_n=5
    )

    print("Keyphrases:")
    for phrase, score in keyphrases:
        print(f"- {phrase}: {score:.4f}")
```

```
===== Keyphrases for HIGH Risk Severity =====

Document 1 excerpt: that the financial markets may experience increased volatility and uncertainty. The financial crisis...
Keyphrases:
- exacerbated: 0.5614
- pandemic: 0.4918
- inflationary: 0.4398
- profitability: 0.3792
- results: 0.3292

Document 2 excerpt: ", and the inability to access or utilize these systems, as well as the inability to use our website...
Keyphrases:
- disruptions: 0.6493
- stringent: 0.4961
- deletion: 0.4803
- profitability: 0.4360
- result: 0.2946

Document 3 excerpt: "Item 8.01. Other Events.

On March 21, 2023, the Company entered into an agreement with its wholly-...
Keyphrases:
- liquidity: 0.6837
- anticipate: 0.6609
- geographies: 0.5877
- antitrust: 0.5359
- f5: 0.5007

Document 4 excerpt: and the impact of the COVID-19 pandemic on our business and financial performance, including the eff...
Keyphrases:
- cybersecurity: 0.6060
- pandemic: 0.5567
- disruptions: 0.5452
- mitigation: 0.4373
- results: 0.3809

Document 5 excerpt: of the Company’s operations, as well as the impact of changes in interest rates on our financial con...
Keyphrases:
- volatility: 0.5828
- downgrade: 0.5243
- baa2: 0.5229
- mitigate: 0.5115
- results: 0.3880

===== Keyphrases for MEDIUM Risk Severity =====

Document 1 excerpt: ...effective income tax rate will continue to be influenced by the geographic mix of income and inco...
Keyphrases:
```

```
- audits: 0.6762
- unrecognized: 0.6569
- optimizing: 0.6362
- 2023: 0.5649
- examinations: 0.3498

Document 2 excerpt: ITEM 8.01 OTHER EVENTS

On June 15, 2023, the Company announced that it has entered into a definitiv...
```

YAKE

We explore YAKE only for comparison with KeyBERT, we expect KeyBERT to yield better results.

```
import yake


# Initialize YAKE key word extractor (Bigrams)
yake_extractor = yake.KeywordExtractor(
    lan="en",
    n=2, # bi-grams
    dedupLim=0.9, # deduplication threshold
    dedupFunc='seqm', #d eduplication function
    windowsSize=1, # window size
    top=5, # number of keywords/keyphrases
    features=None
)


# Extract keyphrases by severity level (Bigrams)
for risk in ['HIGH', 'MEDIUM', 'LOW', 'NONE']:
    print(f"\n===== YAKE Keyphrases for {risk} Risk Severity =====")

    # Sample 5-10 documents randomly from each severity level
    risk_docs = df[df['risk_severity'] == risk]['input'].sample(5).values

    for i, doc in enumerate(risk_docs):
        # Display first 100 characters of the document for aesthetic purposes
        print(f"\nDocument {i+1} excerpt: {doc[:100]}...")

        # Extract keyphrases with YAKE
        # For YAKE lower scores mean more significant (opposite of KeyBERT)
        keyphrases = yake_extractor.extract_keywords(doc)

        print("Keyphrases (lower score = more important):")
        for phrase, score in keyphrases:
            print(f"- {phrase}: {score:.4f}")
```

```
Document 2 excerpt: Item 8.01. Other Events.
```

```
On June 15, 2023, the Company entered into an underwriting agreement (the ...
Keyphrases (lower score = more important):
- Company: 0.0033
- underwriting agreement: 0.0060
- Common Stock: 0.0195
- Current Report: 0.0218
- offering: 0.0262

Document 3 excerpt: "In the table below:

As of December 31, 2022, the Company had $5.4 billion in cash and cash equival...
Keyphrases (lower score = more important):
- Company: 0.0040
- commercial paper: 0.0096
- billion: 0.0211
- commercial: 0.0214
- paper: 0.0267

Document 4 excerpt: "the Company’s business, financial condition, results of operations and cash flows. The Company is e...
Keyphrases (lower score = more important):
- Company: 0.0017
- foreign currency: 0.0107
- foreign: 0.0183
- investment portfolio: 0.0226
- currency: 0.0234

Document 5 excerpt: Item 1.01 Entry into a Material Definitive Agreement.

On June 16, 2022, the Company entered into a ...
Keyphrases (lower score = more important):
- Purchase Agreement: 0.0008
- Material Definitive: 0.0053
- Company: 0.0060
- Purchase: 0.0063
- Agreement: 0.0064

===== YAKE Keyphrases for NONE Risk Severity =====

Document 1 excerpt: Item 8.01. Other Events.

On April 10, 2023, the Company issued a press release announcing that it h...
Keyphrases (lower score = more important):
- Company: 0.0036
- SEC: 0.0116
- Company Board: 0.0125
- Company compliance: 0.0179
- SEC staff: 0.0215

Document 2 excerpt: Item 1.01 Entry into a Material Definitive Agreement

On July 12, 2022, the Company entered into an ...
```

```
# Initialize YAKE key word extractor (Unigrams)
yake_extractor_uni = yake.KeywordExtractor(
    lan="en",
    n=1, # bi-grams
    dedupLim=0.9, # deduplication threshold
```

```
    dedupFunc='seqm', #d eduplication function
    windowsSize=1, # window size
    top=5, # number of keywords/keyphrases
    features=None
)

# Extract keyphrases by severity level (Unigrams)
for risk in ['HIGH', 'MEDIUM', 'LOW', 'NONE']:
    print(f"\n===== YAKE Keyphrases for {risk} Risk Severity =====")

    # Sample 5-10 documents randomly from each severity level
    risk_docs = df[df['risk_severity'] == risk]['input'].sample(5).values

    for i, doc in enumerate(risk_docs):
        # Display first 100 characters of the document for aesthetic purposes
        print(f"\nDocument {i+1} excerpt: {doc[:100]}...")

        # Extract keyphrases with YAKE
        # For YAKE lower scores mean more significant (opposite of KeyBERT)
        keyphrases = yake_extractor_uni.extract_keywords(doc)

        print("Keyphrases (lower score = more important):")
        for phrase, score in keyphrases:
            print(f"- {phrase}: {score:.4f}")
```

```
Document 3 excerpt: a 1% increase in the annual average cost of the commodity, which would result in a 0.5% increase in...
Keyphrases (lower score = more important):
- Plant: 0.0118
- Equipment: 0.0157
- Property: 0.0170
- cost: 0.0347
- balance: 0.0375

Document 4 excerpt: and (b) below, and the accompanying notes to the Consolidated Financial Statements.

As of December ...
Keyphrases (lower score = more important):
- Company: 0.0027
- stock: 0.0092
- Financial: 0.0173
- common: 0.0213
- Losses: 0.0322

Document 5 excerpt: Item 1.01 Entry into a Material Definitive Agreement

On February 10, 2023, the Company entered into...
Keyphrases (lower score = more important):
- Agreement: 0.0011
- stock: 0.0012
- Form: 0.0017
- Exhibit: 0.0035
- restricted: 0.0063
```

Testing (Evaluation, Comparison and Explainability)

```
from sklearn.metrics import confusion_matrix, classification_report, ConfusionMatrixDisplay, accuracy_score, f1_score
import matplotlib.pyplot as plt
import torch
import numpy as np
from torch import nn
from transformers import AutoModelForSequenceClassification, AutoTokenizer
from torch.utils.data import DataLoader, Dataset


# Encoding
severity_mapping = {
    'NONE': 0,
    'LOW': 1,
    'MEDIUM': 2,
    'HIGH': 3
}

# Apply the mapping to create a new target variable
test_df['severity'] = test_df['risk_severity'].map(severity_mapping)
```

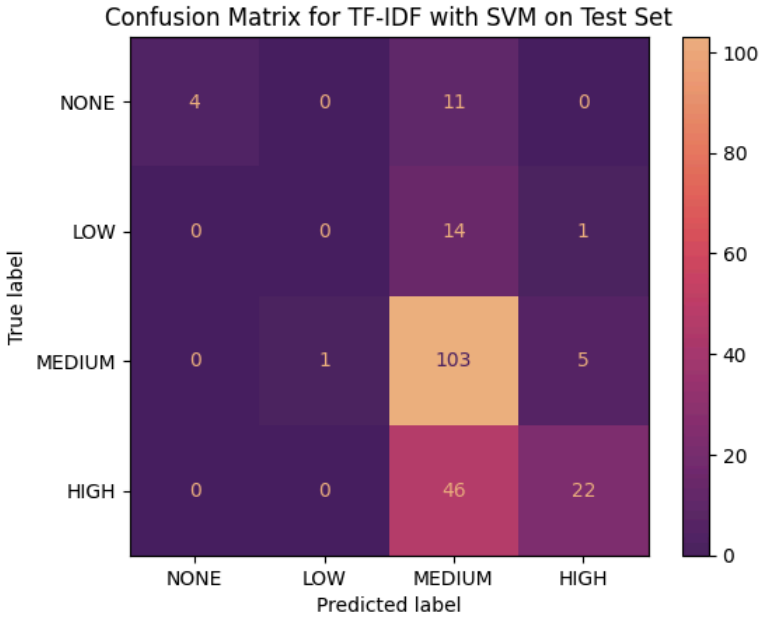
Base Model (TF-IDF + SVM)

```
# Assuming your best model is TF-IDF with SVM
best_model = SVC(class_weight='balanced', random_state=42)  # Use your optimized parameters here
best_model.fit(X_train_tfidf, aug_y_train)  # Train on your augmented training data

# Process test set with the same vectorizer you used for training
X_test_tfidf = tfidf_vectorizer.transform(test_df['input'])
y_test = test_df['severity']  # Using your encoded target variable

# Make predictions on test set
y_pred = best_model.predict(X_test_tfidf)


# Classification report
print("\nClassification Report on Test Set:")
print(classification_report(y_test, y_pred, target_names=class_names))
```

```
Classification Report on Test Set:
              precision    recall  f1-score   support

        NONE       1.00      0.27      0.42        15
         LOW       0.00      0.00      0.00        15
      MEDIUM       0.59      0.94      0.73       109
        HIGH       0.79      0.32      0.46        68

    accuracy                           0.62       207
   macro avg       0.59      0.38      0.40       207
weighted avg       0.64      0.62      0.56       207
```

```
# Create confusion matrix
cm = confusion_matrix(y_test, y_pred)
class_names = ['NONE', 'LOW', 'MEDIUM', 'HIGH']

# Plot confusion matrix
plt.figure(figsize=(10, 8))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
disp.plot(cmap='flare_r', values_format='d')
plt.title('TF-IDF SVM Confusion Matrix on Test Set')
plt.show()
```

```
<Figure size 1000x800 with 0 Axes>
```



Best Model (Finetuned FinBERT)

```
class FinancialRiskDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_length=512):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = str(self.texts[idx])
        label = self.labels[idx]
        encoding = self.tokenizer(
            text,
            truncation=True,
            padding='max_length',
            max_length=self.max_length,
            return_tensors='pt'
        )
        return {
            'input_ids': encoding['input_ids'].squeeze(),
            'attention_mask': encoding['attention_mask'].squeeze(),
```

```
            'labels': torch.tensor(label, dtype=torch.long)
        }


device = 'cuda' if torch.cuda.is_available() else 'cpu'

# Same model used fot Training
def model_fn(dropout_rate=0.3):
    model = AutoModelForSequenceClassification.from_pretrained(
        "ProsusAI/finbert",
        num_labels=4,
        ignore_mismatched_sizes=True
    )
    # Modifying the dropout rate of the classifier
    model.classifier.dropout = nn.Dropout(p=dropout_rate)
    return model

# Tokenizer
tokenizer = AutoTokenizer.from_pretrained("ProsusAI/finbert")
```

```
tokenizer_config.json: 100%  252/252 [00:00<00:00, 5.69kB/s]
config.json: 100%  758/758 [00:00<00:00, 21.2kB/s]
vocab.txt: 100%  232k/232k [00:00<00:00, 5.50MB/s]
special_tokens_map.json: 100%  112/112 [00:00<00:00, 2.75kB/s]
```

```
# Set up the test dataset and dataloader
test_texts = test_df['input'].values
test_labels = test_df['severity'].values
test_dataset = FinancialRiskDataset(test_texts, test_labels, tokenizer)
test_loader = DataLoader(test_dataset, batch_size=8, shuffle=False)


# Load model saved
saved_model_path = "/content/drive/MyDrive/NLP CW/finbert_kfoldmodel3.pt"
checkpoint = torch.load(saved_model_path)
best_model_state = checkpoint['model_state_dict']


# Initialize a fresh model with the same configuration and load the saved state
model = model_fn(dropout_rate=0.3).to(device)
model.load_state_dict(best_model_state)
model.eval()  # Set to evaluation mode
```

Show hidden output

```
# Make predictions on test set
all_preds = []
all_labels = []

with torch.no_grad():
    for batch in test_loader:
```

```
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            labels = batch['labels'].to(device)

            outputs = model(input_ids=input_ids, attention_mask=attention_mask).logits
            _, preds = torch.max(outputs, dim=1)

            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

    # Convert to numpy arrays
    y_pred = np.array(all_preds)
    y_true = np.array(all_labels)


    # Classification report
    class_names = ['NONE', 'LOW', 'MEDIUM', 'HIGH']
    print("\nClassification Report on Test Set:")
    print(classification_report(y_true, y_pred, target_names=class_names))
```

```
    Classification Report on Test Set:
                  precision    recall  f1-score   support

              NONE       0.33      0.07      0.11        15
               LOW       0.17      0.07      0.10        15
            MEDIUM       0.62      0.75      0.68       109
              HIGH       0.65      0.63      0.64        68

          accuracy                           0.61       207
         macro avg       0.44      0.38      0.38       207
      weighted avg       0.58      0.61      0.58       207
```

```
    # Create and plot confusion matrix
    cm = confusion_matrix(y_true, y_pred)

    plt.figure(figsize=(10, 8))
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
    disp.plot(cmap='flare_r', values_format='d')
    plt.title('FinBERT Confusion Matrix on Test Set')
    plt.tight_layout()
    plt.show()
```

<Figure size 1000x800 with 0 Axes>